

МИНИСТЕРСТВО ОБРАЗОВАНИЯ И НАУКИ РОССИЙСКОЙ
ФЕДЕРАЦИИ

САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИНФОРМАЦИОННЫХ
ТЕХНОЛОГИЙ, МЕХАНИКИ И ОПТИКИ

Факультет программной инженерии и компьютерной техники

КУРСОВАЯ РАБОТА

Тема: Разработка настольного приложения на Python для
автоматизированного анализа ЭЭГ-ритмов и оценки состояния покоя по
спектральным признакам

Работу выполнил Касьяненко Вера Михайловна группы Р3420
(фамилия, имя, отчество) (номер группы)

Руководитель Штенников Дмитрий Геннадьевич
(фамилия, имя, отчество)

Работа защищена " _____ " _____ 2026 г.

с оценкой _____

Подписи членов комиссии: _____

СОДЕРЖАНИЕ

1. Введение	3
1.1 Актуальность	3
1.2 Постановка задачи и функциональность приложения	3
1.3 Цель работы	4
1.4 Задачи работы	4
1.5 Объект и предмет исследования	5
1.6 Ожидаемый результат	5
2. Теоретическая часть	5
2.1 Основы ЭЭГ и физиологические предпосылки	5
2.2 Основные ритмы ЭЭГ	5
2.3 Типичные артефакты ЭЭГ и способы снижения их влияния	6
2.4 Анализ методов и алгоритмов обработки ЭЭГ	7
2.5 Фильтрация ЭЭГ-сигнала (ФВЧ, ФНЧ, полосовая и режекторная)	7
2.6 Спектральный анализ (БПФ, PSD и вейвлет-преобразование)	8
2.7 Выделение признаков (Features Extraction)	9
2.8 Сравнение с baseline и вычисление изменений	9
2.9 Обзор существующего ПО для анализа ЭЭГ	9
3. Практическая часть	10
3.1 Требования и сценарий использования	10
3.2 Выбор технологий и обоснование	10
3.3 Архитектура приложения: основные модули	11
3.3.1 Поддерживаемые форматы	13
3.3.2 Удаление дублей времени и ресэмплинг на равномерную сетку	13
3.4 Предобработка сигнала (очистка перед спектральным анализом)	14
3.5 Извлечение спектральных признаков (PSD следует мощности в полосах)	15
3.6 Правила классификации состояния относительно baseline	15
3.7 Визуализация результатов и вкладки интерфейса	16
3.8 Валидация расчетов и оценка производительности	17
3.9 Результаты работы программы	17
3.9.1 Сценарий 1 – Покой	17
3.9.2 Сценарий 2 – Когнитивная активность	20
3.9.3 Сравнение с результатами, полученными через MNE-Python	22
3.9.4 Производительность	22
3.9.5 Эксперименты с данными других людей	23
4. Заключение	25
4.1 Выводы о проделанной работе	25
4.2 Оценка соответствия результата поставленной цели	26
4.3 Перспективы дальнейшего развития проекта	26
5. Список использованных источников	26

1. ВВЕДЕНИЕ

1.1 АКТУАЛЬНОСТЬ

Электроэнцефалография (ЭЭГ) – один из базовых неинвазивных методов регистрации биоэлектрической активности мозга. ЭЭГ широко применяется в клинической диагностике, нейрофизиологии, нейропсихологии, а также в прикладных направлениях, связанных с нейроинтерфейсами и биологической обратной связью.

В учебных лабораторных работах по дисциплине "Нейротехнологии на основе электроэнцефалографии" показано, что спектральные характеристики ЭЭГ закономерно меняются при смене функционального состояния человека. В частности, в состоянии расслабленного бодрствования при закрытых глазах возрастает выраженность альфа-активности, а при когнитивной нагрузке и концентрации внимания чаще наблюдается относительное усиление бета-активности и ослабление альфа-компоненты. Для прикладных задач удобно иметь программный модуль, который автоматически строит "профиль" базового состояния (baseline), затем сравнивает с ним новые записи и выдает количественную оценку изменений и вероятное состояние.

Актуальность разработки такого модуля заключается в том, что он автоматизирует типовой цикл обработки ЭЭГ-записей, повторяющийся в экспериментах: чтение данных, предобработка, спектральный анализ, расчет признаков, сравнение с baseline и интерпретация результата.

1.2 ПОСТАНОВКА ЗАДАЧИ И ФУНКЦИОНАЛЬНОСТЬ ПРИЛОЖЕНИЯ

Разрабатываемое приложение предназначено для автоматизации базового анализа ЭЭГ. Приложение должно выполнять следующий набор действий:

- Загрузка данных ЭЭГ из файла (CSV, EDF, SET/FDT).
- Приведение данных к единому виду: устранение дублей времени, интерполяция на равномерную сетку, установка рабочей частоты дискретизации.
- Предобработка сигнала перед спектральным анализом: удаление постоянной составляющей/дрейфа, подавление выбросов, полосовая фильтрация в диапазоне 1-40 Гц, подавление сетевой помехи 50 Гц.
- Оценка спектральной плотности мощности (PSD) методом Уэлча и вычисление спектральных признаков: общая мощность в диапазоне 1-40 Гц, мощность альфа-

ритма 8-13 Гц, мощность бета-ритма 13-30 Гц, относительная мощность альфа-ритма, относительная мощность бета-ритма, а также отношение мощностей альфа- и бета-ритмов.

- Сравнение признаков текущей записи с baseline и вычисление процентных изменений.
- Классификация состояния ("покой"/"когнитивная активность") по изменению отношений α/β и относительных вкладов α и β .
- Визуализация результатов: сигнал до/после предобработки, PSD, выделение α и β диапазонов, огибающие ритмов, столбчатая диаграмма ключевых изменений.
- Сравнение PSD/мощностей с расчетами MNE-Python.
- Оценка производительности: замеры времени этапов и пикового потребления памяти.

1.3 ЦЕЛЬ РАБОТЫ

Разработать программный модуль на Python для автоматизированного анализа спектральных признаков ЭЭГ и оценки состояния покоя/когнитивной активности путем сравнения текущей записи с baseline-записью в состоянии покоя.

1.4 ЗАДАЧИ РАБОТЫ

1. Проанализировать предметную область: физиологические основы ЭЭГ-ритмов, типовые артефакты и подходы к их подавлению.
2. Выделить набор спектральных признаков, которые устойчиво отражают изменения функционального состояния: мощности ритмов, относительные мощности, индексные отношения.
3. Спроектировать архитектуру модуля: загрузка данных, предобработка, спектральный анализ, извлечение признаков, сравнение с baseline, принятие решения, визуализация.
4. Реализовать на Python: чтение ЭЭГ-файлов; предобработку: обрезку, устранение дубликатов по времени, интерполяцию на равномерную временную сетку, полосовую фильтрацию и подавление сетевой помехи; оценку спектральной плотности мощности и расчет признаков по диапазонам; сравнение признаков с baseline и расчет изменения в процентах; выдачу вероятного состояния: "покой" или "когнитивная активность".

5. Выполнить тестирование модуля на записях лабораторных работ и оценить согласованность результатов с ожидаемыми эффектами (альфа-реактивность, рост бета при нагрузке).

1.5 ОБЪЕКТ И ПРЕДМЕТ ИССЛЕДОВАНИЯ

Объект исследования – ЭЭГ-сигнал человека, зарегистрированный при разных функциональных состояниях в рамках лабораторных работ.

Предмет исследования – методы цифровой обработки ЭЭГ и спектральные признаки, позволяющие сравнивать записи "до" и "после" относительно baseline.

1.6 ОЖИДАЕМЫЙ РЕЗУЛЬТАТ

Результатом работы является настольное приложение с графическим интерфейсом, которое: загружает baseline-запись ЭЭГ (состояние покоя) и вычисляет профиль спектральных признаков; загружает текущую запись и вычисляет тот же набор признаков; сравнивает текущие признаки с baseline, выводит процентные изменения и предполагаемое состояние ("покой"/"когнитивная активность"); отображает графики сигнала до/после предобработки, спектры PSD и вклад диапазонов α и β ; выполняет валидацию вычислений PSD/мощностей и выводит расхождения; фиксирует производительность по этапам (время обработки и пиковое потребление памяти).

2. ТЕОРЕТИЧЕСКАЯ ЧАСТЬ

2.1 Основы ЭЭГ и физиологические предпосылки

ЭЭГ – это регистрация разности потенциалов на поверхности головы, возникающей в результате суммарной активности популяций нейронов, прежде всего пирамидных клеток коры. Регистрируемый сигнал является слабым, подвержен помехам и требует аккуратной фильтрации и анализа.

В ЭЭГ традиционно выделяют частотные диапазоны (ритмы), каждый из которых связан с определенными функциональными состояниями. Для задач данной курсовой важны прежде всего альфа- и бета-диапазоны, но для полноты приводится общий перечень.

2.2 Основные ритмы ЭЭГ

Дельта-ритм: примерно 0,5-4 Гц. Наиболее характерен для глубоких стадий сна, может появляться при выраженной утомляемости, а также в отдельных клинических состояниях.

Тета-ритм: примерно 4-8 Гц. Связан с сонливостью, переходными состояниями, иногда с когнитивной обработкой и рабочей памятью.

Альфа-ритм: примерно 8-13 Гц. Выражен при расслабленном бодрствовании, особенно при закрытых глазах; при умственной активности и внимании часто наблюдается явление "альфа-блокады" – снижение мощности альфа-диапазона.

Бета-ритм: примерно 14-30 Гц (иногда до 40 Гц). Связан с активным бодрствованием, концентрацией, когнитивным контролем. При повышенной мышечной активности высокочастотная часть диапазона может загрязняться ЭМГ-артефактами.

Гамма-активность: выше 30-40 Гц. Заметно чувствительна к мышечным помехам и качеству аппаратуры.

В рамках ваших лабораторных работ дополнительно рассматривались следующие ритмы: каппа-, лямбда-, мю-ритмы. В данной курсовой они могут использоваться как дальнейшее расширение приложения, однако базовая логика классификации "покой/когнитивная активность" опирается на наиболее устойчивые маркеры: альфа и бета.

2.3 Типичные артефакты ЭЭГ и способы снижения их влияния

ЭЭГ-сигнал содержит полезную активность и помехи. Основные артефакты:

Окулографические (моргание, движения глаз) – обычно дают низкочастотные выбросы и дрейф, могут усиливать компоненты в диапазоне до нескольких Гц и искажать спектр.

Мышечные (ЭМГ) – часто выражены в диапазоне 20-100 Гц и выше, особенно при напряжении лба, челюсти, шеи.

Кардиальные (ЭКГ) – могут давать периодические компоненты.

Сетевая помеха – в Европе типично 50 Гц; проявляется как узкий спектральный пик и его гармоники.

Артефакты записи – потери выборок, дубликаты по времени, скачки при начале и завершении активности.

В учебной реализации разумно применять "простые" методы: обрезку нестационарных фрагментов в начале и конце записи; контроль монотонности времени и удаление некорректных шагов; интерполяцию на равномерную сетку; полосовую фильтрацию 0,5-40 Гц для подавления дрейфа и высокочастотного шума; режекторный фильтр 50 Гц при наличии сетевой помехи; простые пороговые критерии выбросов по амплитуде.

2.4 АНАЛИЗ МЕТОДОВ И АЛГОРИТМОВ ОБРАБОТКИ ЭЭГ

Обработка ЭЭГ включает несколько классов методов, выбор которых зависит от постановки задачи (клиническая диагностика, BCI, оценка состояния, поиск событий) и доступных данных (число каналов, наличие разметки, качество записи).

Временные методы: анализ амплитуды, статистик (RMS, дисперсия), детекция выбросов и артефактов по порогам. Эти подходы просты и вычислительно легкие, но плохо отделяют полезную активность от помех без частотного анализа.

Частотные методы: вычисление спектра/PSD (БПФ, метод Уэлча), расчет мощности в диапазонах и индексных признаков (α/β и др.). Для задач оценки функционального состояния частотные признаки являются наиболее устойчивыми и интерпретируемыми, поэтому они выбраны как основа проекта.

Вейвлет-преобразование: позволяет анализировать нестационарные изменения во времени, хорошо подходит для поиска событий и кратковременных паттернов.

Пространственные методы (многоканальные): re-referencing, лапласиан, пространственные фильтры, ICA/ASR.

Машинное обучение: классификация состояния по признакам (логистическая регрессия, SVM, деревья, нейросети), возможно без явного baseline. Такой подход перспективен, но требует датасета разметки и контроля переобучения.

2.5 ФИЛЬТРАЦИЯ ЭЭГ-СИГНАЛА (ФВЧ, ФНЧ, ПОЛОСОВАЯ И РЕЖЕКТОРНАЯ)

ЭЭГ-сигнал содержит как полезные ритмические компоненты, так и помехи (дрейф, мышечный шум, наводки сети). Перед спектральным анализом применяется фильтрация, чтобы ограничить сигнал рабочим диапазоном частот и повысить устойчивость расчетов.

ФВЧ предназначен для подавления медленных составляющих: базового дрейфа, перемещения электродов, потоотделения и низкочастотных артефактов. Эти компоненты искажают спектр, завышают суммарную мощность и могут маскировать изменения альфа/бета. Типичные частоты среза для ЭЭГ – 0,5-1 Гц.

ФНЧ ограничивает высокочастотный шум и частично подавляет мышечные артефакты (ЭМГ), которые особенно заметны на частотах выше 20–30 Гц. Для задач, где ключевые признаки лежат в диапазонах α (8–13 Гц) и β (13–30 Гц), разумно ограничить спектр сверху значением ≈ 40 Гц.

На практике вместо последовательного ФВЧ и ФНЧ часто используют полосовой фильтр с границами $[f_{low}, f_{high}]$, например 1-40 Гц. Это позволяет одновременно:

- убрать дрейф и сверхнизкие частоты, не несущие полезной информации для выбранных признаков;
- ограничить высокочастотный шум и ЭМГ-вклад;
- повысить устойчивость PSD и интегральных мощностей.

Сетевая помеха проявляется узкой полосой около 50 Гц (в некоторых странах 60 Гц) и может вносить паразитную энергию в спектр, а также создавать искажения после дискретизации/аналоговой части тракта. Для ее подавления применяется режекторный фильтр (notch) на частоте f_0 (обычно 50 Гц) с коэффициентом добротности Q , определяющим ширину вырезаемой полосы: чем больше Q , тем уже "провал" фильтра.

2.6 СПЕКТРАЛЬНЫЙ АНАЛИЗ (БПФ, PSD и ВЕЙВЛЕТ-ПРЕОБРАЗОВАНИЕ)

Спектральные признаки удобнее получать не из "одного" быстрого преобразования Фурье (БПФ) всей записи, а через более устойчивую оценку спектральной плотности мощности (PSD), например методом Уэлча. Дискретное преобразование Фурье (ДПФ):

$$X[k] = \sum_{n=0}^{N-1} x[n] * e^{-j2\pi kn/N},$$

где N – число отсчетов в анализируемом окне, k – индекс частотной компоненты. На практике ДПФ вычисляется алгоритмом БПФ, что ускоряет расчет.

Спектр $X[k]$ показывает наличие периодических составляющих, однако при анализе ЭЭГ одного БПФ на всей записи часто недостаточно, ЭЭГ нестационарна: ритмы меняются во времени; единичный спектр чувствителен к выбросам и артефактам; сравнение двух записей по одному спектру может быть нестабильным.

Для устойчивых признаков чаще используют спектральную плотность мощности (Power Spectral Density, PSD), которая оценивает "мощность на частоту".

Поскольку ЭЭГ часто меняется по времени (например, кратковременные всплески β , подавление α при стимуле), так же применяются методы анализа "время-частота". Наиболее распространенный вариант — непрерывное вейвлет-преобразование (CWT):

$$W(a, b) = \int x(t) \psi * \left(\frac{t-b}{a}\right) dt,$$

где a — масштаб (связан с частотой), b — сдвиг по времени, ψ — материнский вейвлет. В отличие от БПФ, вейвлет-преобразование дает локализованное представление: как меняется энергия частотных компонентов во времени.

Преимущества вейвлетов для ЭЭГ: лучше работают с нестационарными сигналами; позволяют выделять кратковременные события и динамику ритмов.

2.7 ВЫДЕЛЕНИЕ ПРИЗНАКОВ (FEATURES EXTRACTION)

Мощность диапазона (band power) – пусть $P_{xx}(f)$ – PSD. Тогда мощность в диапазоне $[f_1, f_2]$:

$$P_{band} = \int_{f_1}^{f_2} P_{xx}(f) df.$$

В дискретной форме интеграл заменяется суммой или численным интегрированием по частотной сетке PSD.

Относительная мощность – общая мощность в анализируемом диапазоне $[f_{min}, f_{max}]$:

$$P_{total} = \int_{f_{min}}^{f_{max}} P_{xx}(f) df, R_{band} = \frac{P_{band}}{P_{total}}$$

Относительная мощность снижает влияние общего усиления сигнала и помогает сравнивать записи.

Отношения – для практического решения "покой/когнитивная активность" полезны индексные признаки:

- отношение альфа к бета:

$$I_{\alpha/\beta} = \frac{P_{\alpha}}{P_{\beta}}$$

- отношение бета к альфа:

$$I_{\beta/\alpha} = \frac{P_{\beta}}{P_{\alpha}}$$

В рамках ваших лабораторных работ ключевой ожидаемый эффект – рост бета и снижение альфа при когнитивной нагрузке по сравнению с состоянием покоя.

2.8 СРАВНЕНИЕ С BASELINE И ВЫЧИСЛЕНИЕ ИЗМЕНЕНИЙ

Пусть для признака F известны значения F_{base} и F_{test} . Тогда процентное изменение:

$$\Delta F(\%) = \frac{F_{test} - F_{base}}{F_{base}} \cdot 100\%.$$

2.9 ОБЗОР СУЩЕСТВУЮЩЕГО ПО ДЛЯ АНАЛИЗА ЭЭГ

EEGLAB – MATLAB-пакет для анализа ЭЭГ, ориентированный на работу с экспериментальными исследованиями. Архитектурно строится вокруг структуры данных EEG (набор каналов, частота дискретизации, события, сегментация), поддерживает расширение через плагины. Сильные стороны: развитая экосистема для ICA-очистки,

работы с событиями/эпохами, визуализация и множество готовых сценариев обработки. Ограничение для проекта: зависимость от MATLAB и избыточность для простого одноканального сравнения записей по PSD.

MNE-Python – библиотека на Python для анализа ЭЭГ/МЭГ с современной объектной моделью данных, поддержкой множества форматов и последовательных пайплайнов обработки. Возможности включают фильтрацию, спектральные оценки, ICA, обработку событий, статистику, визуализацию, а также продвинутое методы (например, источниковая реконструкция при наличии анатомических данных). Для проекта MNE-Python полезна как: средство чтения форматов EDF/SET и эталонная реализация PSD для валидации вычислений.

Brainstorm – MATLAB-ориентированная GUI-среда для ЭЭГ/МЭГ со структурированной "базой исследований" (subjects/studies), удобной визуализацией и пошаговыми пайплайнами обработки. Особо сильна в сценариях многоканального анализа, работе с анатомическими моделями и источниковой активностью.

Вывод по обзору: профессиональные пакеты (EEGLAB/MNE/Brainstorm) решают широкий круг задач, однако для учебного проекта целесообразно реализовать облегченное приложение, которое дает интерпретируемые признаки и простые правила, обеспечивает визуализацию и контроль корректности.

3. ПРАКТИЧЕСКАЯ ЧАСТЬ

3.1 ТРЕБОВАНИЯ И СЦЕНАРИЙ ИСПОЛЬЗОВАНИЯ

Приложение работает в двух режимах:

- Формирование baseline: пользователь передает ЭЭГ-файл, записанный в состоянии покоя. На выходе формируется профиль признаков baseline.
- Оценка нового состояния: пользователь передает новый ЭЭГ-файл. Модуль считывает признаки, сравнивает с baseline, выводит процентные изменения и вероятное состояние: "покой" или "когнитивная активность".

Данные соответствуют лабораторным работам: записи длительностью около 5 минут, формат CSV с колонками времени и амплитуды.

3.2 ВЫБОР ТЕХНОЛОГИЙ И ОБОСНОВАНИЕ

Выбран язык Python, поскольку он удобен для научных вычислений и имеет развитую экосистему обработки сигналов. Используемые библиотеки:

- NumPy – массивы и линейная алгебра;

- Pandas – чтение и подготовка табличных данных;
- SciPy (signal) – фильтрация, спектральные оценки, метод Уэлча;
- Matplotlib – визуализация;
- MNE-Python для чтения EDF и тестирования.

3.3 АРХИТЕКТУРА ПРИЛОЖЕНИЯ: ОСНОВНЫЕ МОДУЛИ

Логика приложения разделена на три крупных блока:

1. Загрузка данных (CSV / EDF / EEGLAB SET) и получение временного ряда.
2. Предобработка сигнала (удаление дрейфа, фильтрация, подавление сетевой наводки).
3. Извлечение признаков + классификация состояния + визуализация результатов.

EEG Baseline Comparator — блок-схема алгоритма (архитектура и взаимодействие модулей)

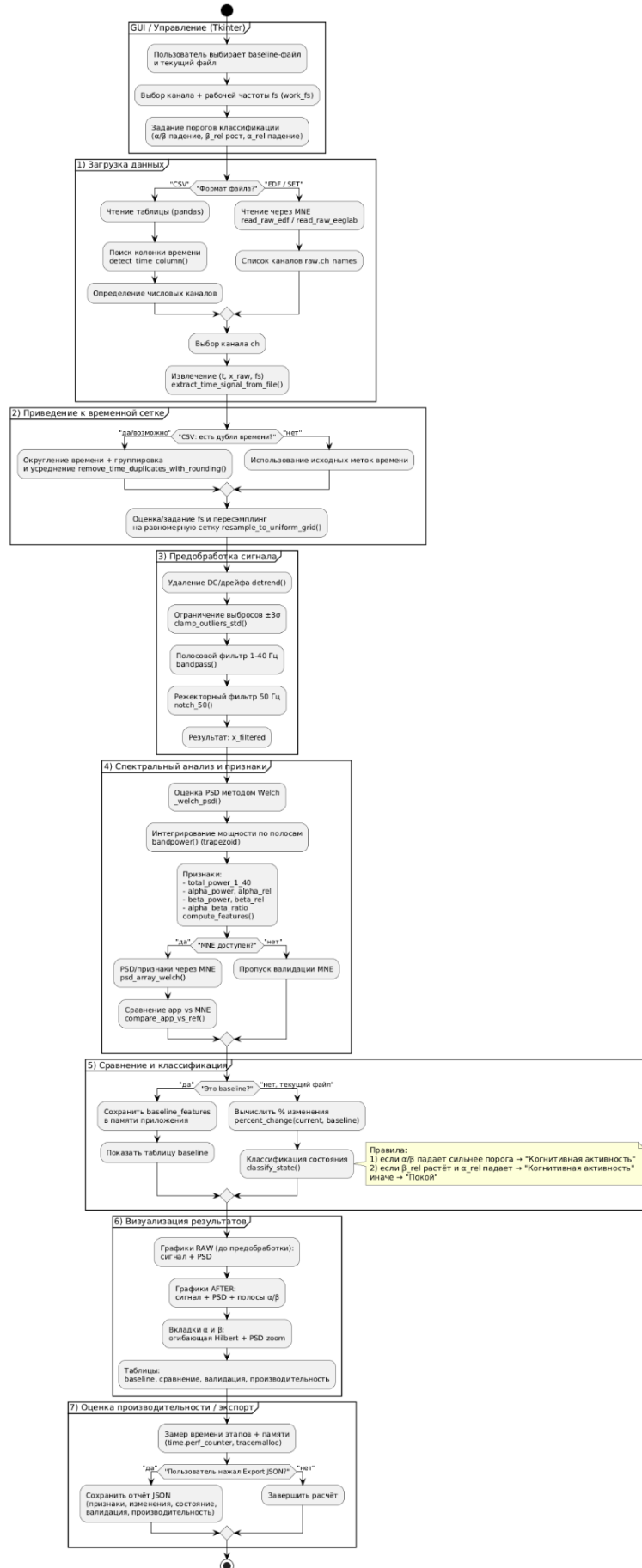


Рис. 1 Блок-схема алгоритма

3.3.1 Поддерживаемые форматы

Поддерживаются: .csv, .edf, .set/.fdt через библиотеку MNE-Python.

```
def load_eeg_file(path: str) -> FileData:
    ext = os.path.splitext(path)[1].lower()

    if ext == ".csv":
        df = _read_csv_robust(path)
        t_col = detect_time_column(df)
        cols = list(df.columns)

        channels = []
        for c in cols:
            if c == t_col:
                continue
            s = pd.to_numeric(df[c], errors="coerce")
            if s.notna().mean() >= 0.7:
                channels.append(c)

        if not channels:
            raise ValueError("Не найдено ни одного числового канала (кроме времени).")

        return FileData(kind="csv", path=path, channels=channels, df=df, t_col=t_col)

    if ext in (".edf", ".set", ".fdt"):
        if not MNE_AVAILABLE:
            raise RuntimeError("Для файлов .edf/.set требуется MNE-Python. Установите: pip install mne")

        if ext == ".edf":
            raw = mne.io.read_raw_edf(path, preload=True, verbose="ERROR")
        else:
            raw = mne.io.read_raw_eeglab(path, preload=True, verbose="ERROR")

        channels = list(raw.ch_names)
        if not channels:
            raise ValueError("Файл загружен, но каналы не найдены.")
        return FileData(kind="mne", path=path, channels=channels, raw=raw)

    raise ValueError(f"Неподдерживаемый формат: {ext}. Поддерживаются: .csv, .edf, .set/.fdt")
```

Рис. 2 Загрузка EEG-файла

Для CSV приложение автоматически ищет колонку времени (`detect_time_column`) и собирает список "похожих на EEG" числовых колонок (каналов). Для EDF/EEGLAB используется MNE, а список каналов берется из `raw.ch_names`.

3.3.2 Удаление дублей времени и ресэмплинг на равномерную сетку

В данных иногда встречаются одинаковые или почти одинаковые временные метки. Чтобы корректно оценить PSD и сравнивать записи между собой, приложение:

1. округляет время,
2. объединяет дубли (усредняет значения),
3. интерполирует сигнал на равномерную временную сетку с частотой `work_fs`.

```

def remove_time_duplicates_with_rounding(df: pd.DataFrame, t_col: str, ch_col: str, round_decimals: int = 6) -> Tuple[np.ndarray, np.ndarray]:
    tmp = df[[t_col, ch_col]].dropna().copy()
    tmp[t_col] = tmp[t_col].astype(float)
    tmp[ch_col] = tmp[ch_col].astype(float)

    tmp["_t_round"] = tmp[t_col].round(round_decimals)

    tmp = (
        tmp.groupby("_t_round", as_index=False)[ch_col]
        .mean()
        .sort_values("_t_round")
        .reset_index(drop=True)
    )

    t = tmp["_t_round"].to_numpy(dtype=float)
    x = tmp[ch_col].to_numpy(dtype=float)
    t = t - t[0]
    return t, x

def resample_to_uniform_grid(t: np.ndarray, x: np.ndarray, target_fs: float) -> Tuple[np.ndarray, np.ndarray]:
    t = np.asarray(t, dtype=float)
    x = np.asarray(x, dtype=float)

    if t.size < 3:
        return t, x

    dt = 1.0 / float(target_fs)
    t_end = float(t[-1])
    if t_end <= 0:
        return t, x

    t_u = np.arange(0.0, t_end + dt, dt)
    x_u = np.interp(t_u, t, x)
    return t_u, x_u

```

Рис. 3 Удаление дублей времени и приведение к равномерной сетке

Ресэмплинг приводит baseline и текущий сигнал к одинаковой частоте дискретизации сигнала (fs), поэтому сравнение α/β становится корректным (иначе PSD и интегрирование по полосам будет несопоставимым).

3.4 ПРЕДОБРАБОТКА СИГНАЛА (ОЧИСТКА ПЕРЕД СПЕКТРАЛЬНЫМ АНАЛИЗОМ)

Перед вычислением спектра сигнал очищается:

1. detrend (удаление постоянной составляющей/дрейфа),
2. ограничение выбросов по правилу $\pm 3\sigma$,
3. полосовой фильтр 1-40 Гц,
4. notch 50 Гц (подавление сетевой помехи).

Фильтрация напрямую влияет на устойчивость решения "покой/активность".

```

def preprocess_wide(x: np.ndarray, fs: float) -> np.ndarray:
    y = detrend(np.asarray(x, dtype=float), type="constant")
    y = clamp_outliers_std(y, k=3.0)
    y = bandpass(y, fs, BAND_WIDE[0], BAND_WIDE[1], order=4)
    y = notch_50(y, fs, notch_hz=50.0, q=30.0)
    return y

```

Рис. 4 Предобработка широкополосного сигнала (preprocess_wide)

3.5 ИЗВЛЕЧЕНИЕ СПЕКТРАЛЬНЫХ ПРИЗНАКОВ (PSD СЛЕДУЕТ МОЩНОСТИ В ПОЛОСАХ)

Ключевая идея: состояние определяется не формой сигнала во времени, а распределением мощности по частотам.

1. Строится PSD методом Welch.
2. Интегрированием PSD по диапазонам получаются мощности:
 - `total_power_1_40` – суммарная мощность 1-40 Гц,
 - `alpha_power` – мощность 8-13 Гц,
 - `beta_power` – мощность 13-30 Гц.
3. Считаются относительные величины:
 - `alpha_rel` = `alpha_power` / `total_power_1_40`,
 - `beta_rel` = `beta_power` / `total_power_1_40`,
 - `alpha_beta_ratio` = `alpha_power` / `beta_power`.

Связывая с "покоем/активностью":

- при покое чаще ожидается относительно более высокий α -вклад => `alpha_rel` выше и `alpha_beta_ratio` выше;
- при когнитивной активности чаще растет β -вклад => `beta_rel` растет, а `alpha_beta_ratio` падает.

```
def compute_features(x: np.ndarray, fs: float) -> Dict[str, float]:
    f, p = _welch_psd(x, fs)

    total = bandpower(f, p, BAND_WIDE[0], min(BAND_WIDE[1], float(np.max(f))))
    a_pow = bandpower(f, p, ALPHA[0], ALPHA[1])
    b_pow = bandpower(f, p, BETA[0], BETA[1])

    a_rel = (a_pow / total) if total > 0 else 0.0
    b_rel = (b_pow / total) if total > 0 else 0.0
    ratio = (a_pow / b_pow) if b_pow > 0 else float("inf")

    return {
        "fs": float(fs),
        "total_power_1_40": float(total),
        "alpha_power": float(a_pow),
        "alpha_rel": float(a_rel),
        "beta_power": float(b_pow),
        "beta_rel": float(b_rel),
        "alpha_beta_ratio": float(ratio),
    }
```

Рис. 5 Вычисление признаков (`compute_features`)

3.6 ПРАВИЛА КЛАССИФИКАЦИИ СОСТОЯНИЯ ОТНОСИТЕЛЬНО BASELINE

Классификация построена как набор простых правил по процентным изменениям признаков относительно baseline:

- если отношение α/β сильно падает, это признак когнитивной активности;
- или если одновременно β_{rel} растет, а α_{rel} падает (оба относительно baseline) – также "когнитивная активность";
- иначе – "покой".

```
def percent_change(new: float, base: float) -> float:
    if base == 0:
        return float("inf") if new != 0 else 0.0
    return float((new - base) / abs(base) * 100.0)

def classify_state(
    baseline: Dict[str, float],
    current: Dict[str, float],
    ratio_drop_pct_thr: float,
    beta_rel_up_pct_thr: float,
    alpha_rel_down_pct_thr: float,
) -> str:
    d_ratio = percent_change(current["alpha_beta_ratio"], baseline["alpha_beta_ratio"])
    d_beta_rel = percent_change(current["beta_rel"], baseline["beta_rel"])
    d_alpha_rel = percent_change(current["alpha_rel"], baseline["alpha_rel"])

    if d_ratio <= -abs(ratio_drop_pct_thr):
        return "Когнитивная активность"
    if (d_beta_rel >= abs(beta_rel_up_pct_thr)) and (d_alpha_rel <= -abs(alpha_rel_down_pct_thr)):
        return "Когнитивная активность"
    return "Покой"
```

Рис. 6 Вычисление процентов и классификация (classify_state)

Пороги задаются в интерфейсе (например: α/β падение = 20%, β_{rel} рост = 20%, α_{rel} падение = 15%). Это удобно, так как можно подстраивать чувствительность классификатора под конкретного человека/датчик.

3.7 ВИЗУАЛИЗАЦИЯ РЕЗУЛЬТАТОВ И ВКЛАДКИ ИНТЕРФЕЙСА

Для интерпретации пользователю нужны не только числа, но и наглядные графики. Поэтому реализованы вкладки:

1. "Общее (после)" – сравнение сигналов после предобработки, PSD после предобработки, и столбчатая диаграмма изменений ключевых метрик (α_{rel} , β_{rel} , α_{β_ratio}).
2. "До предобработки" – исходный сигнал и PSD до фильтрации (чтобы увидеть артефакты/дрейф).
3. " α (8-13)" – огибающая α -ритма по Hilbert и PSD.
4. " β (13-30)" – огибающая β -ритма по Hilbert и PSD.


```
def rhythm_envelope(x: np.ndarray, fs: float, lo: float, hi: float, smooth_sec: float = 0.5) -> np.ndarray:
    y = bandpass(x, fs, lo, hi, order=4)
    env = np.abs(hilbert(y))
    win = int(max(1, round(smooth_sec * fs)))
    env = moving_average(env, win)
    return env
```

Рис. 7 Огибающая ритма через Hilbert (rhythm_envelope)

Огибающая не участвует напрямую в классификации, но помогает визуально подтвердить, что в записи действительно "усиливается" β или "проседает" α .

3.8 ВАЛИДАЦИЯ РАСЧЕТОВ И ОЦЕНКА ПРОИЗВОДИТЕЛЬНОСТИ

Чтобы убедиться, что PSD и мощности считаются корректно, добавлена валидация относительно MNE: приложение параллельно считает PSD через `psd_array_welch` из MNE и выводит процент расхождения.

Также измеряется производительность (время загрузки/ресэмпинга/фильтрации/признаков и пик памяти) через `time.perf_counter()` и `tracemalloc`.

3.9 РЕЗУЛЬТАТЫ РАБОТЫ ПРОГРАММЫ

3.9.1 Сценарий 1 – Покой

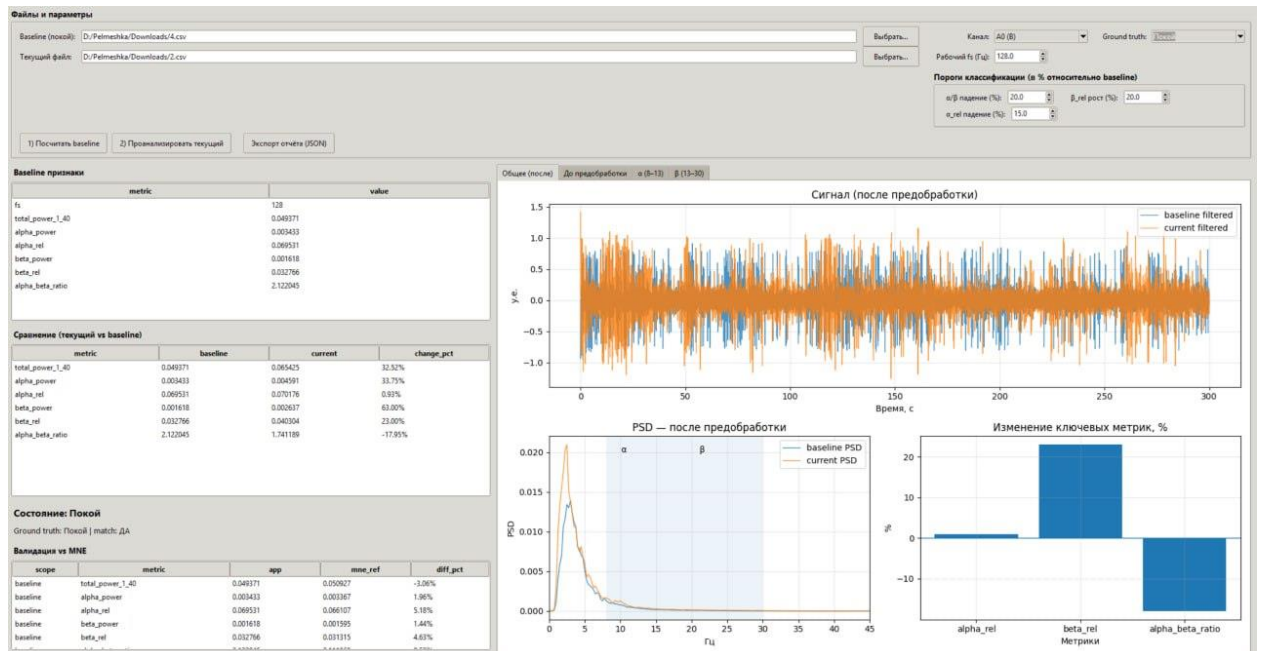


Рис. 8 "Покой": сигнал и PSD после предобработки и изменения метрик

Показаны два сигнала после предобработки (покой с закрытыми глазами – baseline и покой с открытыми глазами – текущий), их PSD и изменения метрик. Условие

классификации "когнитивная активность" не срабатывает, т.к. падение α/β недостаточно велико.

Ключевые изменения относительно baseline (из таблицы сравнения на Рис. 8):

- alpha_rel: +0.93%
- beta_rel: +23.00%
- alpha_beta_ratio: -17.95%

Несмотря на рост beta_rel, ключевой критерий (падение α/β ниже порога) не выполнен, поэтому состояние остается в классе 'покой'. При пороге падения $\alpha/\beta = 20\%$ получаем: $-17.95\% > -20\%$, значит критерий "сильное падение α/β " не выполнен => класс "Покой".

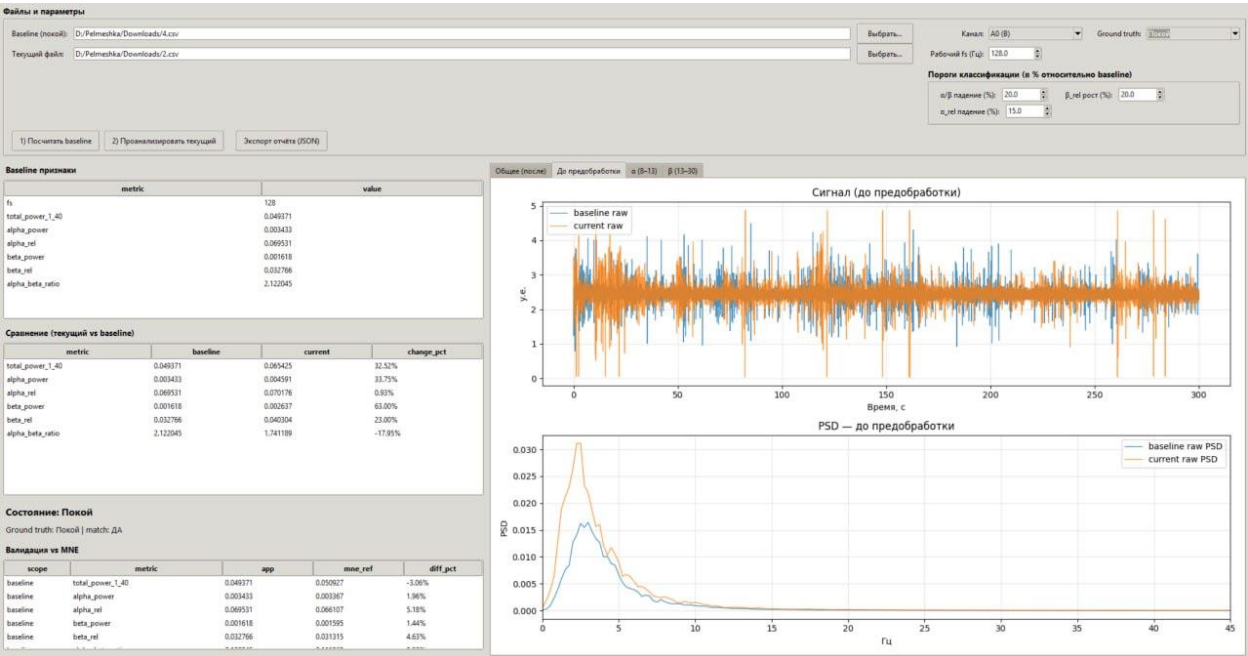


Рис. 9 "Покой": сигнал и PSD до преобработки

Сырой сигнал имеет большой размах и артефакты; PSD до фильтрации отличается по уровню. Этот график показывает работу преобработки перед расчетом α/β .

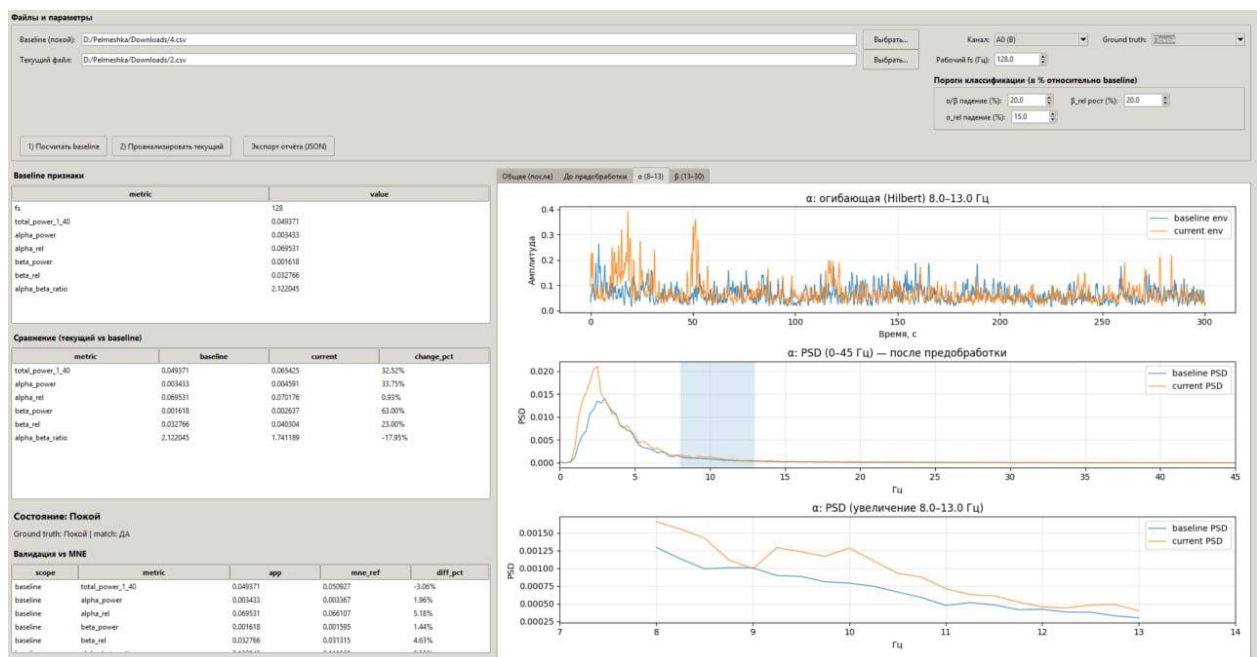


Рис. 10 "Покой": α -диапазон 8-13 Гц (огибающая Hilbert и PSD-увеличение)

Огибающая α -ритма baseline и текущего близка по уровню; на увеличенном PSD в α -полосе спектры сопоставимы.

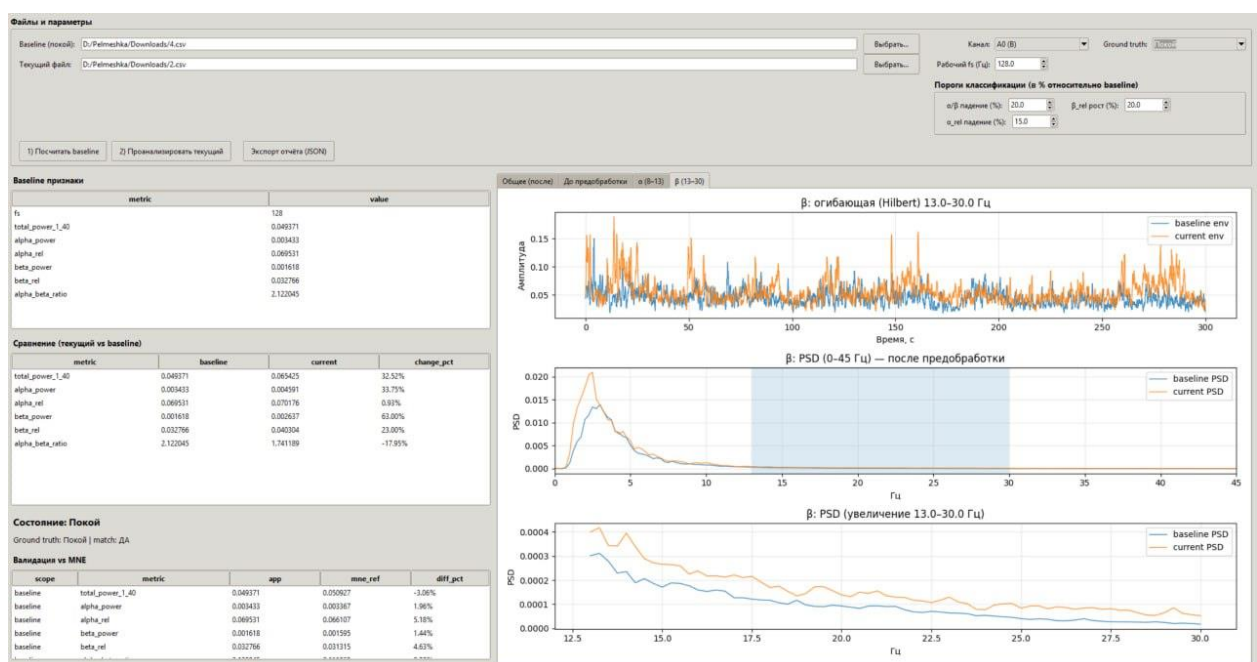


Рис. 11 "Покой": β -диапазон 13-30 Гц (огибающая Hilbert и PSD-увеличение)

β -огибающая и PSD показывают умеренные различия, но они не приводят к падению α/β ниже порога.

3.9.2 Сценарий 2 – Когнитивная активность



Рис. 12 "Когнитивная активность": сигнал и PSD после предобработки и изменения метрик

Сравнивается запись покоя с закрытыми глазами и решение математических задач. Здесь наблюдается типичный паттерн: β -компонента заметно усиливается, α -вклад относительно падает, а отношение α/β резко снижается.

Ключевые изменения относительно baseline (из таблицы сравнения на Рис. 12):

- **alpha_rel: -21.76%**
- **beta_rel: +121.77%**
- **alpha_beta_ratio: -64.72%**

Уже одного критерия достаточно: α/β упал на -64.72%, что существенно ниже порога -20% $\Rightarrow$ состояние уверенно классифицируется как "Когнитивная активность". Дополнительно выполняется и комбинированное правило: β_rel сильно вырос, α_rel упал.



Рис. 13 "Когнитивная активность": сигнал и PSD до преобработки

Сырой сигнал и PSD до фильтрации показывают общую энергетическую картину, но для точного сравнения по α/β все равно используется обработанная версия.

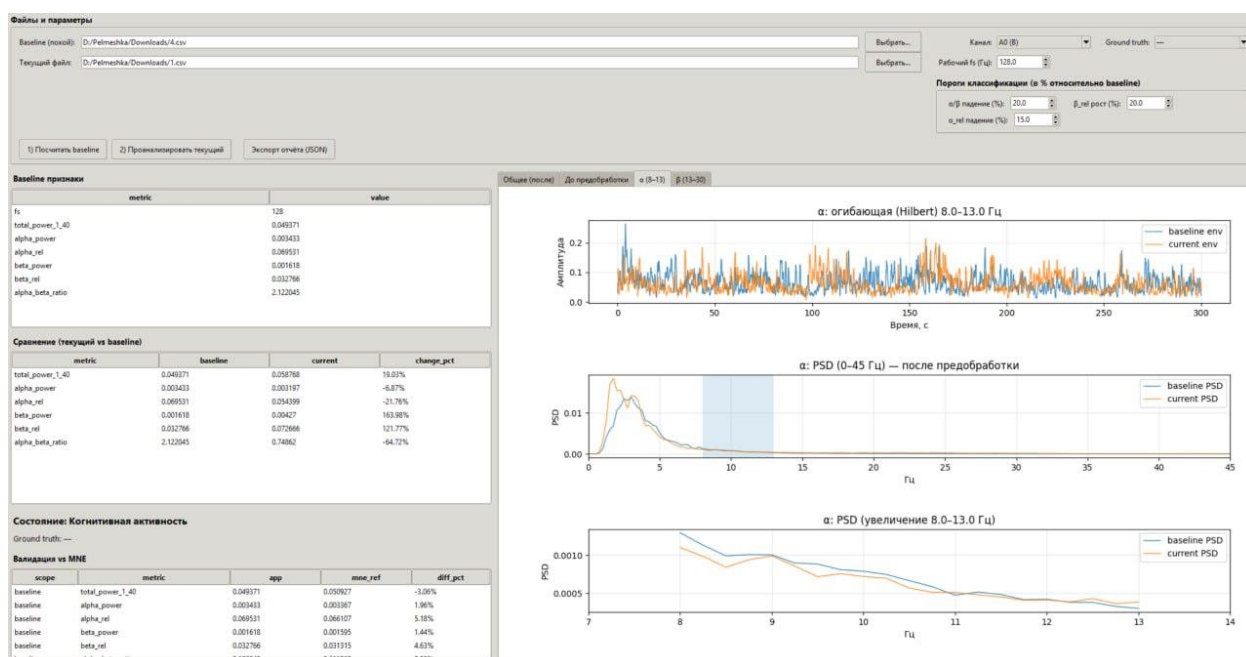


Рис.14 "Когнитивная активность": α -диапазон 8-13 Гц (огибающая Hilbert и PSD-увеличение)

В α -полосе заметно снижение относительного вклада (что согласуется с alpha_rel -21.76%).

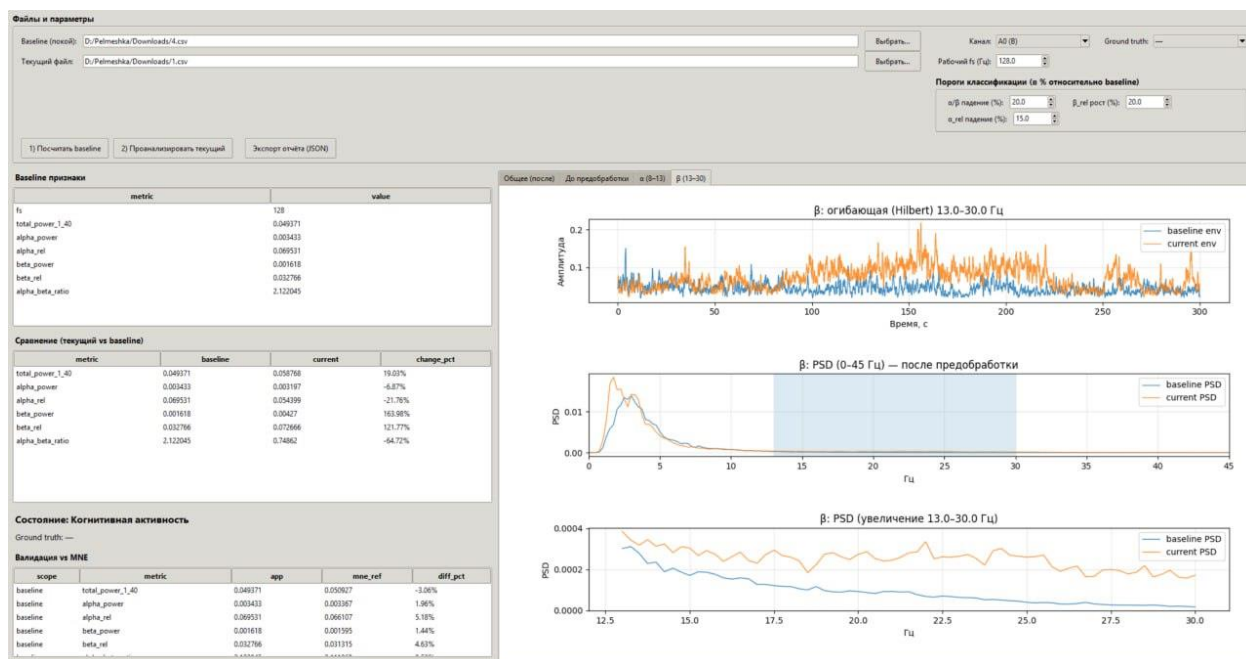


Рис. 15 "Когнитивная активность": β-диапазон 13-30 Гц (огибающая Hilbert и PSD-увеличение)

В β-полосе спектр текущего сигнала выше baseline; это визуально соответствует скачку beta_rel +121.77% и росту β-мощности.

3.9.3 Сравнение с результатами, полученными через MNE-Python

Для валидации корректности реализации спектрального анализа результаты работы модуля сравнивались с оценками, полученными с использованием библиотеки MNE-Python. Для одного и того же сигнала вычислялись PSD методом Уэлча и спектральные признаки (мощности в диапазонах, относительные значения). Относительное расхождение результатов не превышало 3-6%, что подтверждает корректность реализации.

Валидация vs MNE				
scope	metric	app	mne_ref	diff_pct
baseline	total_power_1_40	0.049371	0.050927	-3.06%
baseline	alpha_power	0.003433	0.003367	1.96%
baseline	alpha_rel	0.069531	0.066107	5.18%
baseline	beta_power	0.001618	0.001595	1.44%
baseline	beta_rel	0.032766	0.031315	4.63%

Рис. 16 Сравнение работы приложения с результатами, полученными через MNE-Python

3.9.4 Производительность

Ниже приводятся замеры времени выполнения по этапам и пикового потребления памяти (tracemalloc). Основное время, как правило, уходит на загрузку файла, а вычисления фильтрации/PSD выполняются существенно быстрее.

Производительность		
scope	metric	value
baseline	load_ms	1748.67 ms
baseline	resample_ms	15.54 ms
baseline	preprocess_ms	2.86 ms
baseline	features_ms	12.54 ms
baseline	mne_ref_ms	2.63 ms
baseline	total_ms	1782.47 ms
baseline	peak_mem_kb	32134.2 KB

Рис. 17 Производительность расчета baseline (время этапов и пик памяти)

Пример (по Рис. 17): total_ms $\approx$ 1782.47 ms, peak_mem_kb $\approx$ 32134.2 KB.

Производительность		
scope	metric	value
baseline	load_ms	1748.67 ms
baseline	resample_ms	15.54 ms
baseline	preprocess_ms	2.86 ms
baseline	features_ms	12.54 ms
baseline	mne_ref_ms	2.63 ms
baseline	total_ms	1782.47 ms
baseline	peak_mem_kb	32134.2 KB

Рис. 18 Производительность анализа текущего файла (время этапов и пик памяти)

Пример (по Рис. 18): total_ms $\approx$ 1774.00 ms, peak_mem_kb $\approx$ 32134.4 KB.

3.9.5 Эксперименты с данными других людей

Для дополнительной проверки работоспособности приложения были использованы записи других студентов. Эти данные отличаются условиями регистрации, уровнем шумов и масштабом амплитуд, поэтому служат хорошей проверкой работы приложения.

Для каждого эксперимента baseline формировался из фрагмента, соответствующего "спокойному" состоянию, после чего анализировалась запись с активностью. В интерфейсе также использовалась опция Ground truth (если состояние было известно заранее), чтобы проверить совпадение результата классификации.

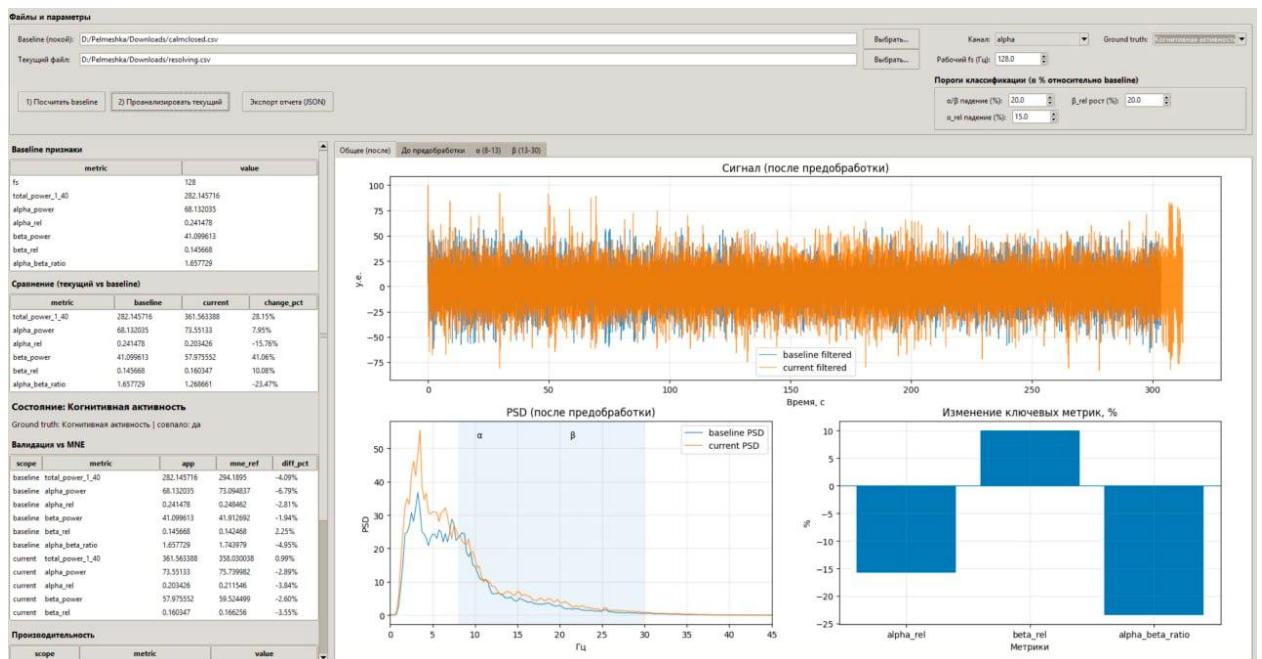


Рис. 19 Публичные данные: сценарий "решение задач" – классификация "Когнитивная активность"

Наблюдается падение отношения α/β и рост β -компоненты относительно baseline:

- alpha_rel: -15.76%
- beta_rel: +10.08%
- alpha_beta_ratio: -23.47%

При пороге падения $\alpha/\beta = 20\%$ условие выполняется ($-23.47\% \leq -20\%$), поэтому запись классифицируется как "когнитивная активность".

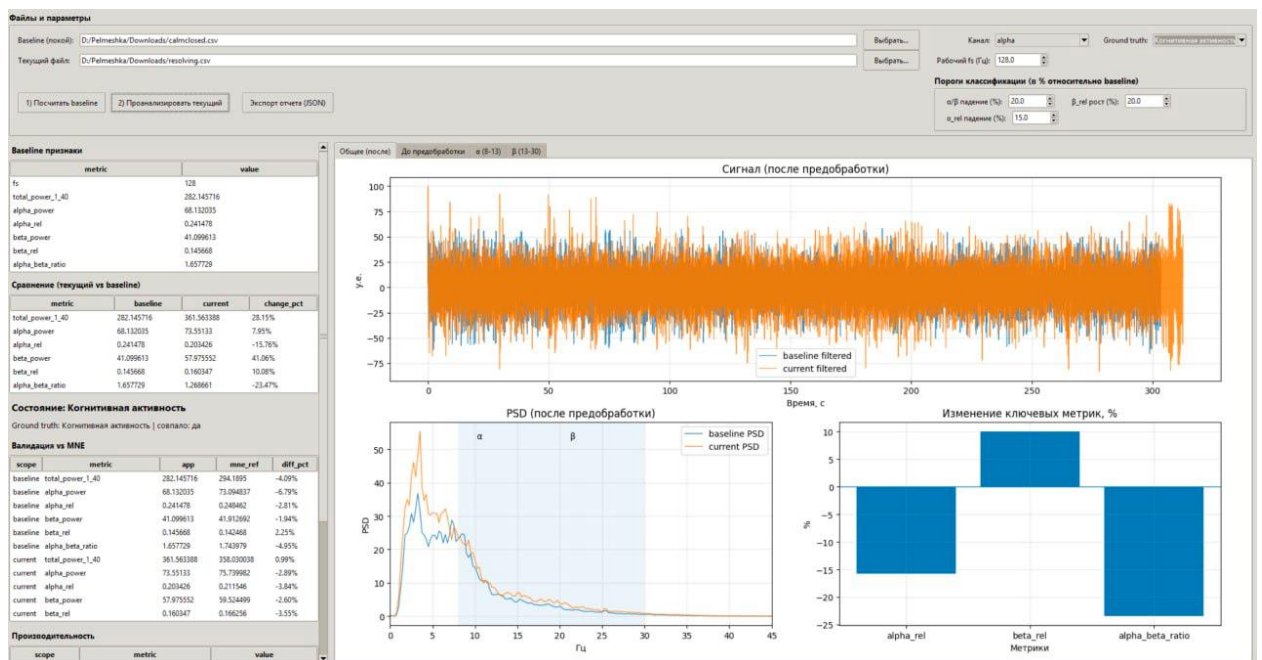


Рис. 20 Публичные данные: сценарий "игра" – классификация "Когнитивная активность".

Эффект выражен сильнее:

- alpha_rel: -22.48%

- beta_rel: +80.30%
- alpha_beta_ratio: -57.01%

Падение α/β значительно превышает порог, дополнительно наблюдается рост beta_rel и снижение alpha_rel, что соответствует ожидаемому паттерну нагрузки.

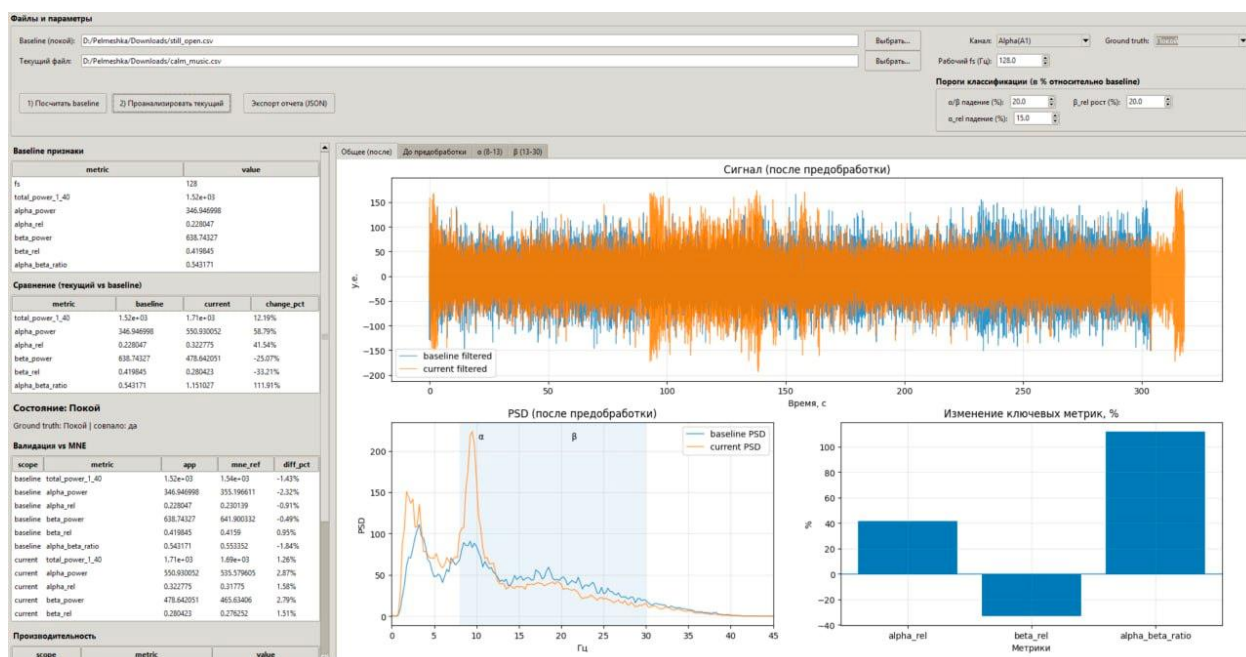


Рис. 21 Публичные данные: сценарий "покой" – классификация "Покой".

Здесь, наоборот, усиливается α -составляющая и растет α/β :

- alpha_rel: +41.54%
- beta_rel: -33.21%
- alpha_beta_ratio: +111.91%

Критерии когнитивной активности не выполняются, поэтому состояние корректно определяется как "покой".

4. ЗАКЛЮЧЕНИЕ

4.1 ВЫВОДЫ О ПРОДЕЛАННОЙ РАБОТЕ

В ходе курсовой работы разработано настольное приложение на Python для анализа ЭЭГ. Реализованы основные этапы обработки: загрузка данных, устранение проблем временной шкалы, ресэмплинг, предобработка (удаление дрейфа, подавление выбросов, полосовая фильтрация 1-40 Гц, режекторная фильтрация 50 Гц), расчет PSD методом Уэлча и извлечение спектральных признаков в диапазонах α и β . Реализована визуализация (сигнал до/после, PSD, выделение диапазонов, огибающие Hilbert) и вывод таблиц сравнения, а также экспорт отчета (JSON).

К основным трудностям можно отнести: неоднородность входных данных (разные разделители, колонки времени, дубли временных меток), необходимость приводить записи к единой частоте дискретизации и обработку шумов на записи.

4.2 ОЦЕНКА СООТВЕТСТВИЯ РЕЗУЛЬТАТА ПОСТАВЛЕННОЙ ЦЕЛИ

Поставленная цель – разработка модуля для автоматизированной оценки состояния "покой/когнитивная активность" по сравнению с baseline – достигнута. На лабораторных данных показано, что при когнитивной нагрузке наблюдаются ожидаемые изменения: рост β -вклада и снижение отношения α/β , что приводит к корректному срабатыванию правил классификации. Дополнительная проверка корректности вычислений выполняется через сравнение с PSD, рассчитанной средствами MNE-Python, а измерения времени и памяти подтверждают практическую применимость решения.

4.3 ПЕРСПЕКТИВЫ ДАЛЬНЕЙШЕГО РАЗВИТИЯ ПРОЕКТА

Масштабирование и пакетная обработка: поддержка анализа не двух, а множества файлов, формирование сводного отчета по серии измерений.

Работа в реальном времени: подключение потока данных от устройства и моментальная классификация.

Переход от правил к ML-классификации: обучение модели (логистическая регрессия/SVM/градиентный бустинг) на признаках, что позволит предсказывать состояние без явного baseline, либо использовать baseline как опцию калибровки. Дополнительно возможна персонализация модели под пользователя.

5. СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Зенков Л. Р. Клиническая электроэнцефалография с элементами эпилептологии. — 10-е изд. — Москва : МЕДпресс-информ, 2023. — 360 с. — ISBN 978-5-907504-83-7.

2. Основы клинической электроэнцефалографии : учебное пособие : [сайт]. — URL: <https://irbis.rmapo.ru/UploadsFilesForIrbis/05672e517bfcc1f92093427fc2ef3c2f.pdf> (дата обращения: 12.01.2026).

3. Кропотов Ю. Д. Количественная электроэнцефалография, когнитивные вызванные потенциалы мозга человека и нейротерапия. — Донецк : Издатель Заславский А. Ю., 2010. — 512 с. — ISBN 978-617-7001-39-2.

4. Нидермайер Э., да Силва Ф. Л. Электроэнцефалография: основы, клиническое применение и смежные области. — [Б. м.] : [б. и.], [б. г.].

5. Отчеты по лабораторным работам №1-№ 6 по дисциплине "Нейротехнологии на основе электроэнцефалографии" (альфа-, бета-, каппа-, лямбда-, мю-ритмы). — [Б. м.] : [б. и.], [б. г.].

6. SciPy Documentation. Signal processing (scipy.signal) : [сайт]. — URL: https://docs.scipy.org/doc/scipy/reference/signal.html (дата обращения: 10.01.2026).

7. SciPy Documentation. `scipy.signal.welch` : [сайт]. — URL: https://docs.scipy.org/doc/scipy/reference/generated/scipy.signal.welch.html (дата обращения: 10.01.2026).

8. NumPy Documentation : [сайт]. — URL: https://numpy.org/doc/stable/ (дата обращения: 10.01.2026).

9. Pandas Documentation : [сайт]. — URL: https://pandas.pydata.org/docs/ (дата обращения: 10.01.2026).

10. Matplotlib Documentation : [сайт]. — URL: https://matplotlib.org/stable/ (дата обращения: 10.01.2026).

11. MNE-Python Documentation : [сайт]. — URL: https://mne.tools/stable/ (дата обращения: 10.01.2026).

ПРИЛОЖЕНИЕ 1

```
import os
import json
import time
import tracemalloc
import numpy as np
import pandas as pd

from dataclasses import dataclass
from typing import Dict, Optional, Tuple, List

from scipy.signal import butter, filtfilt, iirnotch, welch, detrend, hilbert
from scipy.integrate import trapezoid

import tkinter as tk
from tkinter import ttk, filedialog, messagebox

import matplotlib
matplotlib.use("TkAgg")
from matplotlib.figure import Figure
from matplotlib.backends.backend_tkagg import FigureCanvasTkAgg

try:
    import mne
    from mne.time_frequency import psd_array_welch
    MNE_AVAILABLE = True
except Exception:
    mne = None
    psd_array_welch = None
    MNE_AVAILABLE = False

# Signal processing utilities

BAND_WIDE = (1.0, 40.0)
ALPHA = (8.0, 13.0)
BETA = (13.0, 30.0)

METRICS_ORDER = [
    "fs",
    "total_power_1_40",
    "alpha_power",
    "alpha_rel",
    "beta_power",
    "beta_rel",
    "alpha_beta_ratio",
```

```
]

```

```
def _read_csv_robust(path: str) -> pd.DataFrame:
    for sep in [",", ";", "\t"]:
        try:
            df = pd.read_csv(path, sep=sep, engine="python")
            if df.shape[1] >= 2:
                return df
        except Exception:
            pass
    return pd.read_csv(path)

def detect_time_column(df: pd.DataFrame) -> str:
    candidates = ["Время (с)", "Время", "time", "Time", "t", "timestamp"]
    for c in candidates:
        if c in df.columns:
            return c
    return df.columns[0]

def estimate_fs_from_time(t: np.ndarray) -> float:
    t = np.asarray(t, dtype=float)
    if t.size < 3:
        return float("nan")
    dt = np.diff(t)
    dt = dt[dt > 0]
    if dt.size == 0:
        return float("nan")
    return float(1.0 / np.median(dt))

def clamp_outliers_std(x: np.ndarray, k: float = 3.0) -> np.ndarray:
    x = np.asarray(x, dtype=float)
    mu = float(np.mean(x))
    sd = float(np.std(x))
    if sd == 0.0:
        sd = 1.0
    return np.clip(x, mu - k * sd, mu + k * sd)

def lowpass(x: np.ndarray, fs: float, hi: float, order: int = 4) -> np.ndarray:
    x = np.asarray(x, dtype=float)
    if fs <= 0:
        return x
    nyq = 0.5 * fs
```

```

hi = min(float(hi), nyq * 0.99)
if hi <= 0:
    return x
b, a = butter(order, hi / nyq, btype="low")
return filtfilt(b, a, x)

def bandpass(x: np.ndarray, fs: float, lo: float, hi: float, order: int = 4) ->
np.ndarray:
    x = np.asarray(x, dtype=float)
    if fs <= 0:
        return x
    nyq = 0.5 * fs
    lo = max(0.001, float(lo))
    hi = float(hi)
    hi = min(hi, nyq * 0.99)
    if hi <= lo:
        return x
    b, a = butter(order, [lo / nyq, hi / nyq], btype="band")
    return filtfilt(b, a, x)

def notch_50(x: np.ndarray, fs: float, notch_hz: float = 50.0, q: float = 30.0) ->
np.ndarray:
    x = np.asarray(x, dtype=float)
    if fs <= 0:
        return x
    nyq = 0.5 * fs
    if notch_hz >= nyq * 0.99:
        return x
    w0 = notch_hz / nyq
    b, a = iirnotch(w0, q)
    return filtfilt(b, a, x)

def moving_average(x: np.ndarray, win: int) -> np.ndarray:
    if win <= 1:
        return x
    kernel = np.ones(win, dtype=float) / float(win)
    return np.convolve(x, kernel, mode="same")

def rhythm_envelope(x: np.ndarray, fs: float, lo: float, hi: float, smooth_sec: float =
0.5) -> np.ndarray:
    y = bandpass(x, fs, lo, hi, order=4)
    env = np.abs(hilbert(y))
    win = int(max(1, round(smooth_sec * fs)))

```

```

env = moving_average(env, win)
return env

def preprocess_wide(x: np.ndarray, fs: float) -> np.ndarray:
    y = detrend(np.asarray(x, dtype=float), type="constant")
    y = clamp_outliers_std(y, k=3.0)
    y = bandpass(y, fs, BAND_WIDE[0], BAND_WIDE[1], order=4)
    y = notch_50(y, fs, notch_hz=50.0, q=30.0)
    return y

def _nperseg_4sec(fs: float, n: int) -> int:
    nperseg = int(max(256, round(fs * 4.0)))
    nperseg = min(nperseg, n, 4096)
    if nperseg < 8:
        nperseg = min(n, 8)
    return nperseg

def _welch_psd(x: np.ndarray, fs: float) -> Tuple[np.ndarray, np.ndarray]:
    nperseg = _nperseg_4sec(fs, len(x))
    freqs, psd = welch(x, fs=fs, nperseg=nperseg)
    return freqs, psd

def bandpower(freqs: np.ndarray, psd: np.ndarray, lo: float, hi: float) -> float:
    m = (freqs >= lo) & (freqs <= hi)
    if not np.any(m):
        return 0.0
    return float(trapezoid(psd[m], freqs[m]))

def compute_features(x: np.ndarray, fs: float) -> Dict[str, float]:
    f, p = _welch_psd(x, fs)

    total = bandpower(f, p, BAND_WIDE[0], min(BAND_WIDE[1], float(np.max(f))))
    a_pow = bandpower(f, p, ALPHA[0], ALPHA[1])
    b_pow = bandpower(f, p, BETA[0], BETA[1])

    a_rel = (a_pow / total) if total > 0 else 0.0
    b_rel = (b_pow / total) if total > 0 else 0.0
    ratio = (a_pow / b_pow) if b_pow > 0 else float("inf")

    return {
        "fs": float(fs),
        "total_power_1_40": float(total),

```

```

        "alpha_power": float(a_pow),
        "alpha_rel": float(a_rel),
        "beta_power": float(b_pow),
        "beta_rel": float(b_rel),
        "alpha_beta_ratio": float(ratio),
    }

def compute_features_mne_reference(x: np.ndarray, fs: float) -> Optional[Dict[str,
float]]:
    if not MNE_AVAILABLE or psd_array_welch is None:
        return None
    if fs <= 0 or len(x) < 8:
        return None

    nperseg = _nperseg_4sec(fs, len(x))

    psds, freqs = psd_array_welch(
        x[np.newaxis, :].astype(float),
        sfreq=fs, fmin=0.0, fmax=45.0, n_per_seg=nperseg, verbose="ERROR"
    )
    p = psds[0]

    total = bandpower(freqs, p, BAND_WIDE[0], min(BAND_WIDE[1], float(np.max(freqs))))
    a_pow = bandpower(freqs, p, ALPHA[0], ALPHA[1])
    b_pow = bandpower(freqs, p, BETA[0], BETA[1])

    a_rel = (a_pow / total) if total > 0 else 0.0
    b_rel = (b_pow / total) if total > 0 else 0.0
    ratio = (a_pow / b_pow) if b_pow > 0 else float("inf")

    return {
        "fs": float(fs),
        "total_power_1_40": float(total),
        "alpha_power": float(a_pow),
        "alpha_rel": float(a_rel),
        "beta_power": float(b_pow),
        "beta_rel": float(b_rel),
        "alpha_beta_ratio": float(ratio),
    }

def percent_change(new: float, base: float) -> float:
    if base == 0:
        return float("inf") if new != 0 else 0.0
    return float((new - base) / abs(base) * 100.0)

```



```

def classify_state(
    baseline: Dict[str, float],
    current: Dict[str, float],
    ratio_drop_pct_thr: float,
    beta_rel_up_pct_thr: float,
    alpha_rel_down_pct_thr: float,
) -> str:
    d_ratio = percent_change(current["alpha_beta_ratio"], baseline["alpha_beta_ratio"])
    d_beta_rel = percent_change(current["beta_rel"], baseline["beta_rel"])
    d_alpha_rel = percent_change(current["alpha_rel"], baseline["alpha_rel"])

    if d_ratio <= -abs(ratio_drop_pct_thr):
        return "Когнитивная активность"
    if (d_beta_rel >= abs(beta_rel_up_pct_thr)) and (d_alpha_rel <= -
abs(alpha_rel_down_pct_thr)):
        return "Когнитивная активность"
    return "Покой"

def compare_app_vs_ref(app_feats: Dict[str, float], ref_feats: Optional[Dict[str,
float]]) -> Optional[Dict[str, Dict[str, float]]]:
    if ref_feats is None:
        return None
    out: Dict[str, Dict[str, float]] = {}
    for k in METRICS_ORDER:
        if k == "fs":
            continue
        a = float(app_feats.get(k, float("nan")))
        r = float(ref_feats.get(k, float("nan")))
        if np.isnan(a) or np.isnan(r) or np.isinf(a) or np.isinf(r):
            continue
        out[k] = {"app": a, "ref": r, "diff_pct": percent_change(a, r)}
    return out

# Data loading (CSV / EDF / SET)

@dataclass
class FileData:
    kind: str # "csv" | "mne"
    path: str
    channels: List[str]
    df: Optional[pd.DataFrame] = None
    t_col: Optional[str] = None
    raw: Optional["mne.io.BaseRaw"] = None

```

```

def load_eeg_file(path: str) -> FileData:
    ext = os.path.splitext(path)[1].lower()

    if ext == ".csv":
        df = _read_csv_robust(path)
        t_col = detect_time_column(df)
        cols = list(df.columns)

        channels = []
        for c in cols:
            if c == t_col:
                continue

            s = pd.to_numeric(df[c], errors="coerce")
            if s.notna().mean() >= 0.7:
                channels.append(c)

        if not channels:
            raise ValueError("Не найдено ни одного числового канала (кроме времени).")

        return FileData(kind="csv", path=path, channels=channels, df=df, t_col=t_col)

    if ext in (".edf", ".set", ".fdt"):
        if not MNE_AVAILABLE:
            raise RuntimeError("Для файлов .edf/.set требуется MNE-Python. Установите:
pip install mne")

        if ext == ".edf":
            raw = mne.io.read_raw_edf(path, preload=True, verbose="ERROR")
        else:
            raw = mne.io.read_raw_eeglab(path, preload=True, verbose="ERROR")

        channels = list(raw.ch_names)
        if not channels:
            raise ValueError("Файл загружен, но каналы не найдены.")
        return FileData(kind="mne", path=path, channels=channels, raw=raw)

    raise ValueError(f"Неподдерживаемый формат: {ext}. Поддерживаются: .csv, .edf,
.set/.fdt")

def remove_time_duplicates_with_rounding(df: pd.DataFrame, t_col: str, ch_col: str,
round_decimals: int = 6) -> Tuple[np.ndarray, np.ndarray]:
    tmp = df[[t_col, ch_col]].dropna().copy()
    tmp[t_col] = tmp[t_col].astype(float)
    tmp[ch_col] = tmp[ch_col].astype(float)

```

```

tmp["_t_round"] = tmp[t_col].round(round_decimals)

tmp = (
    tmp.groupby("_t_round", as_index=False)[ch_col]
        .mean()
        .sort_values("_t_round")
        .reset_index(drop=True)
)

t = tmp["_t_round"].to_numpy(dtype=float)
x = tmp[ch_col].to_numpy(dtype=float)
t = t - t[0]
return t, x

def resample_to_uniform_grid(t: np.ndarray, x: np.ndarray, target_fs: float) ->
Tuple[np.ndarray, np.ndarray]:
    t = np.asarray(t, dtype=float)
    x = np.asarray(x, dtype=float)

    if t.size < 3:
        return t, x

    dt = 1.0 / float(target_fs)
    t_end = float(t[-1])
    if t_end <= 0:
        return t, x

    t_u = np.arange(0.0, t_end + dt, dt)
    x_u = np.interp(t_u, t, x)
    return t_u, x_u

def extract_time_signal_from_file(fd: FileData, ch_name: str, target_fs: float,
round_decimals: int = 6) -> Tuple[np.ndarray, np.ndarray, float]:
    if fd.kind == "csv":
        assert fd.df is not None and fd.t_col is not None
        t, x = remove_time_duplicates_with_rounding(fd.df, fd.t_col, ch_name,
round_decimals=round_decimals)
        fs_est = estimate_fs_from_time(t)
        if not np.isfinite(fs_est) or fs_est <= 0:
            raise ValueError("Не удалось оценить fs по колонке времени.")
        t_u, x_u = resample_to_uniform_grid(t, x, target_fs=float(target_fs))
        return t_u, x_u, float(target_fs)

    if fd.kind == "mne":
        assert fd.raw is not None

```

```

raw = fd.raw
if ch_name not in raw.ch_names:
    raise ValueError(f"Канал '{ch_name}' не найден в файле.")
t = raw.times.astype(float)
t = t - t[0]
x = raw.get_data(picks=[ch_name])[0].astype(float)
sfreq = float(raw.info.get("sfreq", 0.0))

if sfreq > 0 and target_fs > 0 and sfreq > target_fs:
    aa_hi = min(BAND_WIDE[1], 0.45 * float(target_fs))
    x = lowpass(x, sfreq, aa_hi, order=4)

t_u, x_u = resample_to_uniform_grid(t, x, target_fs=float(target_fs))
return t_u, x_u, float(target_fs)

raise RuntimeError("Неизвестный тип FileData.")

# Result structures

@dataclass
class AnalysisResult:
    baseline_features: Dict[str, float]
    current_features: Dict[str, float]
    changes_pct: Dict[str, float]
    state_label: str
    baseline_ref_features: Optional[Dict[str, float]] = None
    current_ref_features: Optional[Dict[str, float]] = None
    baseline_validation: Optional[Dict[str, Dict[str, float]]] = None
    current_validation: Optional[Dict[str, Dict[str, float]]] = None
    baseline_perf: Optional[Dict[str, float]] = None
    current_perf: Optional[Dict[str, float]] = None
    ground_truth: Optional[str] = None
    gt_match: Optional[bool] = None
    channel: Optional[str] = None

# GUI

class EEGApp(tk.Tk):
    def __init__(self):
        super().__init__()

        self.title("EEG")
        try:
            self.state("zoomed")
        except Exception:

```

```

        self.attributes("-zoomed", True)
self._apply_style()

self.baseline_path: Optional[str] = None
self.current_path: Optional[str] = None

# baseline raw + filtered
self.baseline_t: Optional[np.ndarray] = None
self.baseline_raw: Optional[np.ndarray] = None
self.baseline_x: Optional[np.ndarray] = None
self.baseline_fs: Optional[float] = None
self.baseline_features: Optional[Dict[str, float]] = None
self.baseline_perf: Optional[Dict[str, float]] = None
self.baseline_ref: Optional[Dict[str, float]] = None
self.baseline_val: Optional[Dict[str, Dict[str, float]]] = None

# current raw + filtered
self.current_t: Optional[np.ndarray] = None
self.current_raw: Optional[np.ndarray] = None
self.current_x: Optional[np.ndarray] = None
self.current_fs: Optional[float] = None
self.current_features: Optional[Dict[str, float]] = None
self.current_perf: Optional[Dict[str, float]] = None
self.current_ref: Optional[Dict[str, float]] = None
self.current_val: Optional[Dict[str, Dict[str, float]]] = None

self.channel_candidates: List[str] = []
self.result: Optional[AnalysisResult] = None

self._build_ui()
self._build_plot()

def _apply_style(self):
    style = ttk.Style(self)
    try:
        style.theme_use("clam")
    except Exception:
        pass
    style.configure("TLabel", padding=(2, 2))
    style.configure("TButton", padding=(10, 6))
    style.configure("TLabelframe", padding=(8, 6))
    style.configure("TLabelframe.Label", font=("Segoe UI", 10, "bold"))
    style.configure("Treeview.Heading", font=("Segoe UI", 9, "bold"))
    style.configure("Treeview", rowheight=22)

def _filetypes(self):
    return [

```

```

        ("EEG files", "*.csv *.edf *.set *.fdt"),
        ("CSV", "*.csv"),
        ("EDF", "*.edf"),
        ("EEGLAB SET", "*.set"),
        ("EEGLAB FDT", "*.fdt"),
        ("All files", "*.*"),
    ]

def _build_ui(self):
    root = ttk.Frame(self, padding=10)
    root.pack(fill=tk.BOTH, expand=True)

    top = ttk.LabelFrame(root, text="Файлы и параметры")
    top.pack(side=tk.TOP, fill=tk.X)

    ttk.Label(top, text="Baseline (покой):").grid(row=0, column=0, sticky="w")
    self.baseline_entry = ttk.Entry(top, width=70)
    self.baseline_entry.grid(row=0, column=1, padx=6, sticky="we")
    ttk.Button(top, text="Выбрать...", command=self.pick_baseline).grid(row=0,
column=2, padx=4)

    ttk.Label(top, text="Текущий файл:").grid(row=1, column=0, sticky="w")
    self.current_entry = ttk.Entry(top, width=70)
    self.current_entry.grid(row=1, column=1, padx=6, sticky="we")
    ttk.Button(top, text="Выбрать...", command=self.pick_current).grid(row=1,
column=2, padx=4)

    ttk.Label(top, text="Канал:").grid(row=0, column=3, sticky="e", padx=(20, 4))
    self.ch_var = tk.StringVar(value="")
    self.ch_combo = ttk.Combobox(top, textvariable=self.ch_var, state="readonly",
width=22)
    self.ch_combo.grid(row=0, column=4, sticky="w")

    ttk.Label(top, text="Рабочий fs (Гц):").grid(row=1, column=3, sticky="e",
padx=(20, 4))
    self.work_fs = tk.DoubleVar(value=128.0)
    ttk.Spinbox(top, from_=50.0, to=1000.0, increment=1.0,
textvariable=self.work_fs, width=10).grid(
    row=1, column=4, sticky="w"
)

    ttk.Label(top, text="Ground truth:").grid(row=0, column=5, sticky="e", padx=(20,
4))
    self.gt_var = tk.StringVar(value="-")
    self.gt_combo = ttk.Combobox(top, textvariable=self.gt_var, state="readonly",
width=22,
                                values=["-", "Покой", "Когнитивная активность"])

```

```

self.gt_combo.grid(row=0, column=6, sticky="w")

thr = ttk.LabelFrame(top, text="Пороги классификации (в % относительно
baseline)")
thr.grid(row=2, column=3, columnspan=4, sticky="we", padx=(20, 0), pady=(8, 0))

self.ratio_thr = tk.DoubleVar(value=20.0)
self.beta_up_thr = tk.DoubleVar(value=20.0)
self.alpha_down_thr = tk.DoubleVar(value=15.0)

self._spin(thr, "α/β падение (%) :", self.ratio_thr, 0, 0)
self._spin(thr, "β_rel рост (%) :", self.beta_up_thr, 0, 1)
self._spin(thr, "α_rel падение (%) :", self.alpha_down_thr, 1, 0)

actions = ttk.Frame(top)
actions.grid(row=2, column=0, columnspan=3, sticky="w", pady=(6, 0))
ttk.Button(actions, text="1)              Посчитать              baseline",
command=self.compute_baseline).pack(side=tk.LEFT, padx=4)
ttk.Button(actions, text="2)              Проанализировать        текущий",
command=self.analyze_current).pack(side=tk.LEFT, padx=4)
ttk.Button(actions, text="Экспорт          отчета          (JSON) ",
command=self.export_json).pack(side=tk.LEFT, padx=16)

top.columnconfigure(1, weight=1)

mid = ttk.Frame(root)
mid.pack(side=tk.TOP, fill=tk.BOTH, expand=True, pady=(10, 0))

left = ttk.Frame(mid, width=520)
left.pack(side=tk.LEFT, fill=tk.Y, expand=False)
left.pack_propagate(False)

right = ttk.Frame(mid)
right.pack(side=tk.LEFT, fill=tk.BOTH, expand=True, padx=(10, 0))

left_canvas = tk.Canvas(left, highlightthickness=0)
left_scroll = ttk.Scrollbar(left, orient="vertical", command=left_canvas.yview)
left_canvas.configure(yscrollcommand=left_scroll.set)

left_scroll.pack(side=tk.RIGHT, fill=tk.Y)
left_canvas.pack(side=tk.LEFT, fill=tk.BOTH, expand=True)

left_inner = ttk.Frame(left_canvas)
left_window = left_canvas.create_window((0, 0), window=left_inner, anchor="nw")

def _on_left_configure(event):
    left_canvas.configure(scrollregion=left_canvas.bbox("all"))

```

```

def _on_left_canvas_configure(event):
    left_canvas.itemconfigure(left_window, width=event.width)

left_inner.bind("<Configure>", _on_left_configure)
left_canvas.bind("<Configure>", _on_left_canvas_configure)

def _on_mousewheel(e):
    left_canvas.yview_scroll(int(-1 * (e.delta / 120)), "units")

left_canvas.bind_all("<MouseWheel>", _on_mousewheel)

ttk.Label(left_inner, text="Baseline признаки", font=("Segoe UI", 10,
"bold")).pack(anchor="w")
self.tbl_base = self._make_table(left_inner, height=7)
self.tbl_base.pack(fill=tk.X, pady=(4, 7))

ttk.Label(left_inner, text="Сравнение (текущий vs baseline)", font=("Segoe UI",
10, "bold")).pack(anchor="w")
self.tbl_cmp = self._make_table(left_inner, height=6, with_change=True)
self.tbl_cmp.pack(fill=tk.X, pady=(4, 6))

self.state_label = ttk.Label(left_inner, text="Состояние: -", font=("Segoe UI",
12, "bold"))
self.state_label.pack(anchor="w", pady=(2, 0))

self.gt_label = ttk.Label(left_inner, text="Ground truth: -", font=("Segoe UI",
10))
self.gt_label.pack(anchor="w", pady=(2, 8))

ttk.Label(left_inner, text="Валидация vs MNE", font=("Segoe UI", 10,
"bold")).pack(anchor="w")
self.tbl_val = self._make_val_table(left_inner, height=11)
self.tbl_val.pack(fill=tk.X, pady=(4, 10))

ttk.Label(left_inner, text="Производительность", font=("Segoe UI", 10,
"bold")).pack(anchor="w")
self.tbl_perf = self._make_perf_table(left_inner, height=14)
self.tbl_perf.pack(fill=tk.X, pady=(4, 0))

self.plot_container = right

def _spin(self, parent, label, var, r, c):
    frame = ttk.Frame(parent)
    frame.grid(row=r, column=c, padx=6, pady=2, sticky="w")
    ttk.Label(frame, text=label).pack(side=tk.LEFT)

```



```

        sp = ttk.Spinbox(frame, from_=0.0, to=200.0, increment=1.0, textvariable=var,
width=8)
        sp.pack(side=tk.LEFT, padx=6)

    def _make_table(self, parent, height=10, with_change=False):
        cols = ("metric", "value") if not with_change else ("metric", "baseline",
"current", "change_pct")
        tree = ttk.Treeview(parent, columns=cols, show="headings", height=height)
        for col in cols:
            tree.heading(col, text=col)
            tree.column(col, width=180 if col != "metric" else 240, anchor="w")
        return tree

    def _make_val_table(self, parent, height=8):
        cols = ("scope", "metric", "app", "mne_ref", "diff_pct")
        tree = ttk.Treeview(parent, columns=cols, show="headings", height=height)
        widths = {"scope": 80, "metric": 220, "app": 120, "mne_ref": 120, "diff_pct":
100}
        for col in cols:
            tree.heading(col, text=col)
            tree.column(col, width=widths[col], anchor="w")
        return tree

    def _make_perf_table(self, parent, height=8):
        cols = ("scope", "metric", "value")
        tree = ttk.Treeview(parent, columns=cols, show="headings", height=height)
        widths = {"scope": 80, "metric": 180, "value": 140}
        for col in cols:
            tree.heading(col, text=col)
            tree.column(col, width=widths[col], anchor="w")
        return tree

    def _clear_table(self, tree: ttk.Treeview):
        for item in tree.get_children():
            tree.delete(item)

# Plot (tabs)

    def _build_plot(self):
        self.notebook = ttk.Notebook(self.plot_container)
        self.notebook.pack(fill=tk.BOTH, expand=True)

        self.tab_main = ttk.Frame(self.notebook)
        self.notebook.add(self.tab_main, text="Общее (после)")

        self.tab_raw = ttk.Frame(self.notebook)
        self.notebook.add(self.tab_raw, text="До предобработки")

```

```

self.tab_alpha = ttk.Frame(self.notebook)
self.notebook.add(self.tab_alpha, text="α (8-13)")

self.tab_beta = ttk.Frame(self.notebook)
self.notebook.add(self.tab_beta, text="β (13-30)")

# MAIN filtered
self.fig_main = Figure(figsize=(7.8, 5.4), dpi=100)
self.ax_sig = self.fig_main.add_subplot(2, 2, (1, 2))
self.ax_psd = self.fig_main.add_subplot(2, 2, 3)
self.ax_bar = self.fig_main.add_subplot(2, 2, 4)
self.canvas_main = FigureCanvasTkAgg(self.fig_main, master=self.tab_main)
self.canvas_main.get_tk_widget().pack(fill=tk.BOTH, expand=True)

# RAW
self.fig_raw = Figure(figsize=(7.8, 5.4), dpi=100)
self.ax_raw_sig = self.fig_raw.add_subplot(211)
self.ax_raw_psd = self.fig_raw.add_subplot(212)
self.canvas_raw = FigureCanvasTkAgg(self.fig_raw, master=self.tab_raw)
self.canvas_raw.get_tk_widget().pack(fill=tk.BOTH, expand=True)

# Alpha
self.fig_alpha = Figure(figsize=(7.8, 5.4), dpi=100)
self.ax_alpha_env = self.fig_alpha.add_subplot(311)
self.ax_alpha_psd_full = self.fig_alpha.add_subplot(312)
self.ax_alpha_psd_zoom = self.fig_alpha.add_subplot(313)
self.canvas_alpha = FigureCanvasTkAgg(self.fig_alpha, master=self.tab_alpha)
self.canvas_alpha.get_tk_widget().pack(fill=tk.BOTH, expand=True)

# Beta
self.fig_beta = Figure(figsize=(7.8, 5.4), dpi=100)
self.ax_beta_env = self.fig_beta.add_subplot(311)
self.ax_beta_psd_full = self.fig_beta.add_subplot(312)
self.ax_beta_psd_zoom = self.fig_beta.add_subplot(313)
self.canvas_beta = FigureCanvasTkAgg(self.fig_beta, master=self.tab_beta)
self.canvas_beta.get_tk_widget().pack(fill=tk.BOTH, expand=True)

self.fig_main.tight_layout()
self.fig_raw.tight_layout()
self.fig_alpha.tight_layout()
self.fig_beta.tight_layout()

self._reset_plots()

def _reset_plots(self):
    # main

```

```

self.ax_sig.clear()
self.ax_psd.clear()
self.ax_bar.clear()

self.ax_sig.set_title("Сигнал (после предобработки)")
self.ax_sig.set_xlabel("Время, с")
self.ax_sig.set_ylabel("у.е.")
self.ax_sig.grid(alpha=0.3)

self.ax_psd.set_title("PSD (после предобработки)")
self.ax_psd.set_xlabel("Гц")
self.ax_psd.set_ylabel("PSD")
self.ax_psd.grid(alpha=0.3)
self.ax_psd.set_xlim(0, 45)

self.ax_bar.set_title("Изменение ключевых метрик, %")
self.ax_bar.set_xlabel("Метрики")
self.ax_bar.set_ylabel("%")
self.ax_bar.grid(alpha=0.3)

self.canvas_main.draw()

# raw
self.ax_raw_sig.clear()
self.ax_raw_psd.clear()

self.ax_raw_sig.set_title("Сигнал (до предобработки)")
self.ax_raw_sig.set_xlabel("Время, с")
self.ax_raw_sig.set_ylabel("у.е.")
self.ax_raw_sig.grid(alpha=0.3)

self.ax_raw_psd.set_title("PSD (до предобработки)")
self.ax_raw_psd.set_xlabel("Гц")
self.ax_raw_psd.set_ylabel("PSD")
self.ax_raw_psd.grid(alpha=0.3)
self.ax_raw_psd.set_xlim(0, 45)

self.canvas_raw.draw()

def init_rhythm_axes(ax_env, ax_full, ax_zoom, title, band):
    ax_env.clear()
    ax_full.clear()
    ax_zoom.clear()

    ax_env.set_title(f"{title}: огибающая {band[0]}-{band[1]} Гц")
    ax_env.set_xlabel("Время, с")
    ax_env.set_ylabel("Амплитуда")

```

```

ax_env.grid(alpha=0.3)

ax_full.set_title(f"{title}: PSD (после предобработки)")
ax_full.set_xlabel("Гц")
ax_full.set_ylabel("PSD")
ax_full.grid(alpha=0.3)
ax_full.set_xlim(0, 45)

ax_zoom.set_title(f"{title}: PSD ({band[0]}-{band[1]} Гц)")
ax_zoom.set_xlabel("Гц")
ax_zoom.set_ylabel("PSD")
ax_zoom.grid(alpha=0.3)
ax_zoom.set_xlim(band[0] - 1, band[1] + 1)

init_rhythm_axes(self.ax_alpha_env, self.ax_alpha_psd_full,
self.ax_alpha_psd_zoom, "α", ALPHA)
init_rhythm_axes(self.ax_beta_env, self.ax_beta_psd_full, self.ax_beta_psd_zoom,
"β", BETA)

self.fig_alpha.tight_layout()
self.canvas_alpha.draw()
self.fig_beta.tight_layout()
self.canvas_beta.draw()

def _update_plots(self):
    self._reset_plots()

def plot_psd(ax, x, fs, label):
    f, p = _welch_psd(x, fs)
    ax.plot(f, p, linewidth=1.0, alpha=0.85, label=label)

# RAW plots
if self.baseline_t is not None and self.baseline_raw is not None:
    self.ax_raw_sig.plot(self.baseline_t, self.baseline_raw, linewidth=0.9,
alpha=0.8, label="baseline raw")
if self.current_t is not None and self.current_raw is not None:
    self.ax_raw_sig.plot(self.current_t, self.current_raw, linewidth=0.9,
alpha=0.8, label="current raw")

if (self.baseline_raw is not None) or (self.current_raw is not None):
    self.ax_raw_sig.legend()

if self.baseline_raw is not None and self.baseline_fs is not None:
    plot_psd(self.ax_raw_psd, self.baseline_raw, self.baseline_fs, "baseline raw
PSD")

if self.current_raw is not None and self.current_fs is not None:

```

```

        plot_psd(self.ax_raw_psd, self.current_raw, self.current_fs, "current raw
PSD")

    self.ax_raw_psd.legend()
    self.canvas_raw.draw()

    # MAIN filtered plots
    if self.baseline_t is not None and self.baseline_x is not None:
        self.ax_sig.plot(self.baseline_t, self.baseline_x, linewidth=0.9, alpha=0.8,
label="baseline filtered")
    if self.current_t is not None and self.current_x is not None:
        self.ax_sig.plot(self.current_t, self.current_x, linewidth=0.9, alpha=0.8,
label="current filtered")
    if (self.baseline_x is not None) or (self.current_x is not None):
        self.ax_sig.legend()

    if self.baseline_x is not None and self.baseline_fs is not None:
        plot_psd(self.ax_psd, self.baseline_x, self.baseline_fs, "baseline PSD")
    if self.current_x is not None and self.current_fs is not None:
        plot_psd(self.ax_psd, self.current_x, self.current_fs, "current PSD")

    for (lo, hi, name) in [(ALPHA[0], ALPHA[1], " $\alpha$ "), (BETA[0], BETA[1], " $\beta$ ")]:
        self.ax_psd.axvspan(lo, hi, alpha=0.08)
        self.ax_psd.text((lo + hi) / 2, 0.95, name,
transform=self.ax_psd.get_xaxis_transform(),
                        ha="center", va="top")
    self.ax_psd.legend()

    if self.result is not None:
        keys = ["alpha_rel", "beta_rel", "alpha_beta_ratio"]
        vals = [self.result.changes_pct.get(k, 0.0) for k in keys]
        self.ax_bar.bar(keys, vals)
        self.ax_bar.axhline(0, linewidth=1.0)

    self.canvas_main.draw()

    # Alpha/Beta tabs
    def plot_rhythm(ax_env, ax_full, ax_zoom, band):
        if self.baseline_t is not None and self.baseline_x is not None and
self.baseline_fs is not None:
            env_b = rhythm_envelope(self.baseline_x, self.baseline_fs, band[0],
band[1], smooth_sec=0.5)
            ax_env.plot(self.baseline_t, env_b, linewidth=1.0, alpha=0.85,
label="baseline env")

            f1, p1 = _welch_psd(self.baseline_x, self.baseline_fs)
            ax_full.plot(f1, p1, linewidth=1.0, alpha=0.85, label="baseline PSD")
            ax_full.axvspan(band[0], band[1], alpha=0.08)

```

```

        m = (f1 >= band[0]) & (f1 <= band[1])
        ax_zoom.plot(f1[m], p1[m], linewidth=1.0, alpha=0.9, label="baseline
PSD")

        if self.current_t is not None and self.current_x is not None and
self.current_fs is not None:
            env_c = rhythm_envelope(self.current_x, self.current_fs, band[0],
band[1], smooth_sec=0.5)
            ax_env.plot(self.current_t, env_c, linewidth=1.0, alpha=0.85,
label="current env")

            f2, p2 = _welch_psd(self.current_x, self.current_fs)
            ax_full.plot(f2, p2, linewidth=1.0, alpha=0.85, label="current PSD")
            ax_full.axvspan(band[0], band[1], alpha=0.08)

            m = (f2 >= band[0]) & (f2 <= band[1])
            ax_zoom.plot(f2[m], p2[m], linewidth=1.0, alpha=0.9, label="current
PSD")

        ax_env.legend()
        ax_full.legend()
        ax_zoom.legend()

        plot_rhythm(self.ax_alpha_env, self.ax_alpha_psd_full, self.ax_alpha_psd_zoom,
ALPHA)
        self.canvas_alpha.draw()

        plot_rhythm(self.ax_beta_env, self.ax_beta_psd_full, self.ax_beta_psd_zoom,
BETA)
        self.canvas_beta.draw()

# Perf helper

def _measure_run(self, fn):
    tracemalloc.start()
    t0 = time.perf_counter()
    result, perf = fn()
    total_ms = (time.perf_counter() - t0) * 1000.0
    cur, peak = tracemalloc.get_traced_memory()
    tracemalloc.stop()
    perf = dict(perf)
    perf["total_ms"] = total_ms
    perf["peak_mem_kb"] = peak / 1024.0
    return result, perf

# File picking

```

```

def _apply_default_channel(self, channels: List[str]):
    self.ch_combo["values"] = channels
    self.ch_var.set(channels[0] if channels else "")

def pick_baseline(self):
    path = filedialog.askopenfilename(
        title="Выберите baseline файл (.csv/.edf/.set)",
        filetypes=self._filetypes()
    )
    if not path:
        return
    self.baseline_path = path
    self.baseline_entry.delete(0, tk.END)
    self.baseline_entry.insert(0, path)

    try:
        fd = load_eeg_file(path)
        self.channel_candidates = fd.channels
        self._apply_default_channel(fd.channels)
    except Exception as e:
        messagebox.showerror("Ошибка", f"Не удалось загрузить baseline:\n{e}")

def pick_current(self):
    path = filedialog.askopenfilename(
        title="Выберите текущий файл (.csv/.edf/.set)",
        filetypes=self._filetypes()
    )
    if not path:
        return
    self.current_path = path
    self.current_entry.delete(0, tk.END)
    self.current_entry.insert(0, path)

    try:
        fd = load_eeg_file(path)
        if not self.channel_candidates:
            self.channel_candidates = fd.channels
            self._apply_default_channel(fd.channels)
    except Exception as e:
        messagebox.showerror("Ошибка", f"Не удалось загрузить текущий файл:\n{e}")

# Baseline computation

def compute_baseline(self):
    if not self.baseline_path:
        messagebox.showwarning("Нет файла", "Выберите baseline файл.")

```

```

        return

ch = self.ch_var.get().strip()
if not ch:
    messagebox.showwarning("Нет канала", "Выберите канал.")
    return

try:
    work_fs = float(self.work_fs.get())
    if work_fs <= 1:
        raise ValueError("Рабочий fs должен быть > 1 Гц.")

    def _job():
        perf = {}
        t_load0 = time.perf_counter()
        fd = load_eeg_file(self.baseline_path)
        perf["load_ms"] = (time.perf_counter() - t_load0) * 1000.0

        if ch not in fd.channels:
            raise ValueError(f"Канал '{ch}' отсутствует в baseline файле.")

        t_rs0 = time.perf_counter()
        t, x_raw, fs = extract_time_signal_from_file(fd, ch, target_fs=work_fs,
round_decimals=6)
        perf["resample_ms"] = (time.perf_counter() - t_rs0) * 1000.0

        t_pp0 = time.perf_counter()
        x = preprocess_wide(x_raw, fs)
        perf["preprocess_ms"] = (time.perf_counter() - t_pp0) * 1000.0

        t_ft0 = time.perf_counter()
        feats = compute_features(x, fs)
        perf["features_ms"] = (time.perf_counter() - t_ft0) * 1000.0

        t_ref0 = time.perf_counter()
        ref = compute_features_mne_reference(x, fs)
        perf["mne_ref_ms"] = (time.perf_counter() - t_ref0) * 1000.0

        val = compare_app_vs_ref(feats, ref)
        return (t, x_raw, x, fs, feats, ref, val), perf

    (t, x_raw, x, fs, feats, ref, val), perf = self._measure_run(_job)

    self.baseline_t, self.baseline_raw, self.baseline_x, self.baseline_fs = t,
x_raw, x, fs
    self.baseline_features = feats
    self.baseline_ref = ref

```



```

self.baseline_val = val
self.baseline_perf = perf

self.current_t = None
self.current_raw = None
self.current_x = None
self.current_fs = None
self.current_features = None
self.current_ref = None
self.current_val = None
self.current_perf = None
self.result = None

self._fill_baseline_table(feats)
self._clear_table(self.tbl_cmp)
self.state_label.config(text="Состояние: -")
self.gt_label.config(text="Ground truth: -")
self._fill_validation_table()
self._fill_perf_table()
self._update_plots()
except Exception as e:
    messagebox.showerror("Ошибка baseline", str(e))

# Current analysis

def analyze_current(self):
    if not self.baseline_features:
        messagebox.showwarning("Нет baseline", "Сначала посчитайте baseline.")
        return
    if not self.current_path:
        messagebox.showwarning("Нет файла", "Выберите текущий файл.")
        return

    ch = self.ch_var.get().strip()
    if not ch:
        messagebox.showwarning("Нет канала", "Выберите канал.")
        return

    try:
        work_fs = float(self.work_fs.get())
        if work_fs <= 1:
            raise ValueError("Рабочий fs должен быть > 1 Гц.")

    def _job():
        perf = {}
        t_load0 = time.perf_counter()
        fd = load_eeg_file(self.current_path)

```

```

perf["load_ms"] = (time.perf_counter() - t_load0) * 1000.0

if ch not in fd.channels:
    raise ValueError(f"Канал '{ch}' отсутствует в текущем файле.")

t_rs0 = time.perf_counter()
t, x_raw, fs = extract_time_signal_from_file(fd, ch, target_fs=work_fs,
round_decimals=6)
perf["resample_ms"] = (time.perf_counter() - t_rs0) * 1000.0

t_pp0 = time.perf_counter()
x = preprocess_wide(x_raw, fs)
perf["preprocess_ms"] = (time.perf_counter() - t_pp0) * 1000.0

t_ft0 = time.perf_counter()
feats = compute_features(x, fs)
perf["features_ms"] = (time.perf_counter() - t_ft0) * 1000.0

t_ref0 = time.perf_counter()
ref = compute_features_mne_reference(x, fs)
perf["mne_ref_ms"] = (time.perf_counter() - t_ref0) * 1000.0

val = compare_app_vs_ref(feats, ref)
return (t, x_raw, x, fs, feats, ref, val), perf

(t, x_raw, x, fs, feats, ref, val), perf = self._measure_run(_job)

self.current_t, self.current_raw, self.current_x, self.current_fs = t, x_raw,
x, fs

self.current_features = feats
self.current_ref = ref
self.current_val = val
self.current_perf = perf

changes = {}
for k in self.baseline_features.keys():
    if k == "fs":
        continue
    changes[k] = percent_change(feats.get(k, float("nan")),
self.baseline_features.get(k, float("nan")))

state = classify_state(
    self.baseline_features,
    feats,
    ratio_drop_pct_thr=float(self.ratio_thr.get()),
    beta_rel_up_pct_thr=float(self.beta_up_thr.get()),
    alpha_rel_down_pct_thr=float(self.alpha_down_thr.get()),

```

```

    )

    gt = self.gt_var.get()
    gt_norm = None if gt == "-" else gt
    gt_match = None if gt_norm is None else (gt_norm == state)

    self.result = AnalysisResult(
        baseline_features=self.baseline_features,
        current_features=feats,
        changes_pct=changes,
        state_label=state,
        baseline_ref_features=self.baseline_ref,
        current_ref_features=ref,
        baseline_validation=self.baseline_val,
        current_validation=val,
        baseline_perf=self.baseline_perf,
        current_perf=perf,
        ground_truth=gt_norm,
        gt_match=gt_match,
        channel=ch,
    )

    self._fill_compare_table(self.result)
    self.state_label.config(text=f"Состояние: {state}")
    if gt_norm is None:
        self.gt_label.config(text="Ground truth: -")
    else:
        self.gt_label.config(text=f"Ground truth: {gt_norm} | совпало: {'да' if
gt_match else 'нет'}")

    self._fill_validation_table()
    self._fill_perf_table()
    self._update_plots()

except Exception as e:
    messagebox.showerror("Ошибка анализа", str(e))

# Tables

def _fill_baseline_table(self, feats: Dict[str, float]):
    self._clear_table(self.tbl_base)
    for k in METRICS_ORDER:
        if k in feats:
            self.tbl_base.insert("", tk.END, values=(k, self._fmt(feats[k])))

def _fill_compare_table(self, res: AnalysisResult):
    self._clear_table(self.tbl_cmp)

```

```

keys = [k for k in METRICS_ORDER if k != "fs"]
for k in keys:
    b = res.baseline_features.get(k, float("nan"))
    c = res.current_features.get(k, float("nan"))
    d = res.changes_pct.get(k, float("nan"))
    self.tbl_cmp.insert("", tk.END, values=(k, self._fmt(b), self._fmt(c),
self._fmt_pct(d)))

def _fill_validation_table(self):
    self._clear_table(self.tbl_val)

    if not MNE_AVAILABLE:
        self.tbl_val.insert("", tk.END, values=("-", "MNE недоступен", "-", "-", "-
"))

        return

    if self.baseline_val:
        for metric, row in self.baseline_val.items():
            self.tbl_val.insert("", tk.END, values=(
                "baseline", metric, self._fmt(row["app"]), self._fmt(row["ref"]),
self._fmt_pct(row["diff_pct"])
            ))

        else:
            self.tbl_val.insert("", tk.END, values=("baseline", "нет данных", "-", "-",
"-"))

    if self.current_val:
        for metric, row in self.current_val.items():
            self.tbl_val.insert("", tk.END, values=(
                "current", metric, self._fmt(row["app"]), self._fmt(row["ref"]),
self._fmt_pct(row["diff_pct"])
            ))

        else:
            self.tbl_val.insert("", tk.END, values=("current", "нет данных", "-", "-",
"-"))

def _fill_perf_table(self):
    self._clear_table(self.tbl_perf)

def add(scope: str, perf: Optional[Dict[str, float]]):
    if not perf:
        self.tbl_perf.insert("", tk.END, values=(scope, "нет данных", "-"))
        return

    order = ["load_ms", "resample_ms", "preprocess_ms", "features_ms",
"mne_ref_ms", "total_ms", "peak_mem_kb"]
    for k in order:
        if k in perf:

```

```

        v = perf[k]
        if k.endswith("_ms"):
            self.tbl_perf.insert("", tk.END, values=(scope, k, f"{v:.2f}"
ms"))

            elif k == "peak_mem_kb":
                self.tbl_perf.insert("", tk.END, values=(scope, k, f"{v:.1f}"
KB"))

            else:
                self.tbl_perf.insert("", tk.END, values=(scope, k,
self._fmt(v)))

    add("baseline", self.baseline_perf)
    add("current", self.current_perf)

def _fmt(self, x: float) -> str:
    if x is None:
        return "-"

    if isinstance(x, (float, np.floating)) and (np.isnan(x) or np.isinf(x)):
        return "-"

    if abs(float(x)) >= 1000:
        return f"{float(x):.3g}"
    return f"{float(x):.6f}".rstrip("0").rstrip(".")

def _fmt_pct(self, x: float) -> str:
    if x is None:
        return "-"

    if isinstance(x, (float, np.floating)) and (np.isnan(x) or np.isinf(x)):
        return "-"
    return f"{float(x):.2f}%"

# Export

def export_json(self):
    if self.result is None:
        messagebox.showinfo("Нет отчета", "Сначала выполните анализ текущего файла.")
        return

    path = filedialog.asksaveasfilename(
        title="Сохранить отчет",
        defaultextension=".json",
        filetypes=[("JSON", "*.json")]
    )
    if not path:
        return

    payload = {
        "topic": "EEG",

```

```

        "mne_available": bool(MNE_AVAILABLE),
        "work_fs": float(self.work_fs.get()),
        "channel": self.result.channel,
        "thresholds_pct": {
            "ratio_drop_pct": float(self.ratio_thr.get()),
            "beta_rel_up_pct": float(self.beta_up_thr.get()),
            "alpha_rel_down_pct": float(self.alpha_down_thr.get()),
        },
        "baseline_path": self.baseline_path,
        "current_path": self.current_path,
        "baseline_features": self.result.baseline_features,
        "current_features": self.result.current_features,
        "changes_pct": self.result.changes_pct,
        "state": self.result.state_label,
        "ground_truth": self.result.ground_truth,
        "gt_match": self.result.gt_match,
        "validation_vs_mne": {
            "baseline_ref_features": self.result.baseline_ref_features,
            "current_ref_features": self.result.current_ref_features,
            "baseline_validation": self.result.baseline_validation,
            "current_validation": self.result.current_validation,
        },
        "performance": {
            "baseline": self.result.baseline_perf,
            "current": self.result.current_perf,
        }
    }

}

try:
    with open(path, "w", encoding="utf-8") as f:
        json.dump(payload, f, ensure_ascii=False, indent=2)
    messagebox.showinfo("Готово", f"Отчет сохранен:\n{path}")
except Exception as e:
    messagebox.showerror("Ошибка сохранения", str(e))

if __name__ == "__main__":
    app = EEGApp()
    app.mainloop()

```

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ
УНИВЕРСИТЕТ ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ, МЕХАНИКИ И
ОПТИКИ**

ЗАДАНИЕ НА КУРСОВОЙ ПРОЕКТ (РАБОТУ)

Студент Касьяненко Вера Михайловна
(Фамилия, И., О.)

Факультет ПИиКТ Группа Р3420

Руководитель Штенников Д.Г., СПб НИУ ИТМО, факультет ПИиКТ, доцент
(Фамилия, И., О., место работы, должность)

Дисциплина Нейротехнологии на основе электроэнцефалографии

Наименование темы Разработка программного модуля на Python для
автоматизированного анализа ЭЭГ-ритмов и оценки состояния покоя по
спектральным признакам

Задание Спроектировать и реализовать настольное приложение для
автоматизированной обработки ЭЭГ: загрузка записей, предобработка,
спектральный анализ и классификация состояния «покой/когнитивная
активность» с визуализацией результатов и оформлением пояснительной
записки.

Студент _____ Дата « _____ » _____ 2026 г.
Подпись Дата

Руководитель _____ Дата « _____ » _____ 2026 г.
Подпись Дата

САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ
УНИВЕРСИТЕТ ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ, МЕХАНИКИ И
ОПТИКИ

ОТЗЫВ РУКОВОДИТЕЛЯ О ВЫПОЛНЕНИИ
КУРСОВОЙ РАБОТЫ

Студент _____ Касьяненко Вера Михайловна _____

(Фамилия, И., О.)

Факультет ПИиКТ Группа Р3420

Руководитель Штенников Д.Г., СПб НИУ ИТМО, факультет ПИиКТ, доцент

(Фамилия, И., О., место работы, должность)

Дисциплина Нейротехнологии на основе электроэнцефалографии

Наименование темы Разработка программного модуля на Python для
автоматизированного анализа ЭЭГ-ритмов и оценки состояния покоя по
спектральным признакам

ОЦЕНКА КУРСОВОГО ПРОЕКТА (РАБОТЫ)

№ п/п	Показатели	Оценка			
		5	4	3	0
1.	Способность к работе с литературными источниками, справочной литературой, Интернет-ресурсами и т. п.				
2.	Использование иностранных источников				
3.	Способность к анализу и обобщению информационного материала				
4.	Владение базовыми знаниями в профессиональной области				
5.	Владение базовыми знаниями в смежных областях				
6.	Владение навыками решения технических задач				
7.	Способность применять знания на практике				

№ п/п	Показатели	Оценка			
		5	4	3	0
8.	Уровень и корректность использования в работе методов численного моделирования, инженерных расчетов и статистической обработки данных				
9.	Владение навыками использования современных пакетов компьютерных программ и технологий				
10.	Владение навыками оформления отчетных материалов с применением современных пакетов программ				
11.	Качество оформления пояснительной записки (общий уровень грамотности, стиль изложения, качество иллюстраций, корректность цитирования и пр.**)				
12.	Качество оформления презентации				
13.	Владение навыками публичного выступления и межперсональной коммуникации				
14.	Владение навыками планирования и управления временем при выполнении работы				
Итоговая оценка					

* - не оценивается (трудно оценить)

** согласно рекомендациям

Отмеченные достоинства: _____

Отмеченные недостатки: _____

Заключение: _____

Студент _____ Дата « _____ » _____ 2026 г.

Подпись

Дата

Руководитель _____ Дата « _____ » _____ 2026 г.

Подпись

Дата

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ
УНИВЕРСИТЕТ ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ, МЕХАНИКИ И
ОПТИКИ**

АННОТАЦИЯ НА КУРСОВОЙ ПРОЕКТ (РАБОТУ)

Студент Касьяненко Вера Михайловна
(Фамилия, И., О.)

Факультет ПИиКТ Группа Р3420

Руководитель Штенников Д.Г., СПб НИУ ИТМО, факультет ПИиКТ, доцент
(Фамилия, И., О., место работы, должность)

Дисциплина Нейротехнологии на основе электроэнцефалографии

Наименование темы Разработка настольного приложения на Python для
автоматизированного анализа ЭЭГ-ритмов и оценки состояния покоя по
спектральным признакам

ХАРАКТЕРИСТИКА КУРСОВОГО ПРОЕКТА (РАБОТЫ)

1. Цели и задачи работы

- ☐ Сформулированы при участии студента
- ☐ Предложены студентом
- ☐ Определены руководителем

Цель: разработать программное приложение на Python для анализа ЭЭГ по
спектральным признакам и определения состояния «покой/когнитивная
активность» на основе сравнения с baseline.

Задачи: изучить основы ЭЭГ, ритмы и типовые артефакты; выбрать и реализовать предобработку; реализовать спектральный анализ и расчет признаков; реализовать правило классификации и визуализацию результатов; провести тестирование на лабораторных данных и выполнить оценку производительности.

2. Характер работы

☐

Расчет

☐

Конструирование

☐

Моделирование

☐

Другое, _____

Работа носит прикладной (проектно-исследовательский) характер

3. Содержание работы Работа состоит из разделов: введение, теоретическая часть (основы ЭЭГ, артефакты, методы фильтрации и спектрального анализа, признаки), практическая часть (архитектура приложения, реализация модулей загрузки данных/предобработки/анализа/визуализации, тестирование на лабораторных и внешних данных, валидация и производительность), заключение, список источников, приложение с фрагментами кода и примерами результатов.

4. Выводы

По результатам выполнения курсовой работы разработано настольное приложение для анализа ЭЭГ, реализующее полный цикл обработки: загрузка данных, приведение к равномерной временной сетке, предобработка, расчет PSD и спектральных признаков, классификация состояния. Полученные результаты на лабораторных данных согласуются с ожидаемыми эффектами. Реализованы средства проверки корректности расчетов и замеры производительности по этапам обработки, что подтверждает практическую применимость решения.

Студент _____ Дата « ____ » _____ 2026 г.
Подпись Дата

Руководитель _____ Дата « ____ » _____ 2026 г.
Подпись Дата

САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ
УНИВЕРСИТЕТ ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ, МЕХАНИКИ И
ОПТИКИ

ГРАФИК ВЫПОЛНЕНИЯ КУРСОВОЙ РАБОТЫ

Студент _____ Касьяненко Вера Михайловна _____
(Фамилия, И., О.)

Факультет ПИиКТ Группа Р3420

Руководитель Штенников Д.Г., СПб НИУ ИТМО, факультет ПИиКТ, доцент
(Фамилия, И., О., место работы, должность)

Дисциплина Нейротехнологии на основе электроэнцефалографии

Наименование темы Разработка настольного приложения на Python для
автоматизированного анализа ЭЭГ-ритмов и оценки состояния покоя по
спектральным признакам

№ п/ п	Наименование этапа	Дата завершения		Оценка и подпись руководителя
		Планируе мая	Фактическая	
1	Получение и уточнение задания			
2	Изучение предметной области			
3	Проектирование архитектуры			
4	Реализация предобработки сигнала и спектрального анализа			
5	Реализация интерфейса и визуализации результатов			
6	Тестирование и анализ результатов			
7	Оформление пояснительной записки			

Студент _____ Дата « ____ » _____ 2026 г.
Подпись Дата

Руководитель _____ Дата « ____ » _____ 2026 г.
Подпись Дата